1. CONTENTS.

1. CONTENTS 1
2. ABSTRACT 5
3. INTRODUCTION 6
4. META-ASSEMBLERS 8

4.1. The Need for Meta-Assemblers 8
4.2. Features of Meta-Assemblers 9
4.3. Meta-Assembler Classification 10
4.4. Assembly Language 13

4.4.1. Assembly Language in Context 13
4.4.2. Assembly Language Structure 15

4.5. Machine Language Formats 17

4.5.1. Design Criteria in Machine Instructions 17
4.5.2. Layout of Machine Languages 19

4.6. The Assembly Process 23

4.6.1. Lexical Analysis 25
4.6.2. Syntax Analysis 27
4.6.3. Semantic Analysis 27
4.6.4. Code Generation 29
4.6.5. A Simple Two-Pass Assembler 29

4.7. Implementations 33

4.7.1. Early Implementations 33
4.7.2. CASS-8: A Universal Cross-Assembler 36
4.7.3. A General Purpose Cross-Assembler for Microprocessors 38
4.7.4. UOPCA - A Table Driven UCA Written in BASIC 41

4.7.5. XMETA - A High Level Language Based System for Cross-Assembler Definition 43
4.7.6. MAUFI - A Meta-Assembler with a User Friendly Interface 51
4.7.7. A Generative Approach to Universal Cross-Assembler Design 54

5. A SAMPLE IMPLEMENTATION 57

5.1. Problem Definition 57
5.2. System Analysis 57

5.2.1. Approach to Solution 58
5.2.2. Selection of Programming Language 59

5.3. System Design 61

5.3.1. Decomposition into Modules 61
5.3.2. A General Model for Assembly Languages 64
5.3.3. The IEEE Proposed Assembly Language Standard 67

5.4. Detailed Design 73

5.4.1. Modula-2 Support for Object Oriented Programming .. 74
5.4.2. Design of Individual Modules 76
5.4.3. Character String Handling Module (Strings) 76
5.4.4. Exception Handler Module (Exceptions) 77
5.4.5. Lexical Scanner Module (Lex) 78
5.4.6. Expression Evaluation Module (Expression) 80
5.4.7. Input/Output Interface Module (Interface) 82
5.4.8. Listing File Control Module (Listing) 83
5.4.9. Object File Control Module (ObjectGenerator) 85
5.4.10. Table Handler Module (Table, TableTrees, TableExt) 85
5.4.11. Assembler Definition Module (ADM) 90
5.4.12. ADM for the Motorola MC68000 94
5.4.13. Coordination Module (M2Assem) 99

5.4.14. Assembler Directives Module (PseudoOps) 100

5.5. Implementation 100
5.6. Testing 100

6. CONCLUSIONS 104

6.1. Usefulness of Meta-Assemblers 104
6.2. Evaluation of Implementation 105

6.2.1. Overview 105
6.2.2. Speed Of Assembly 105
6.2.3. Development Time for a New ADM 105
6.2.4. Immediate Improvements 107
6.2.5. Future Developments 108

7. REFERENCES/BIBLIOGRAPHY 110

A. APPENDIX ..... Appendix 1

A.1. Definition Modules ..... Appendix 1

A.1.1. ADM.DEF ..... Appendix 2
A.1.2. Exceptions.DEF ..... Appendix 4
A.1.3. Expression.DEF ..... Appendix 5
A.1.4. Interface.DEF ..... Appendix 6
A.1.5. Lex.DEF ..... Appendix 10
A.1.6. Listing.DEF ..... Appendix 12
A.1.7. Location.DEF ..... Appendix 15
A.1.8. ObjectGenerator.DEF ..... Appendix 16
A.1.9. PseudoOps.DEF ..... Appendix 18
A.1.10. Strings.DEF ..... Appendix 19
A.1.11. Table.DEF ..... Appendix 22
A.1.12. TableExt.DEF ..... Appendix 24
A.1.13. TableTrees.DEF ..... Appendix 26

A.2. Implementation Modules ..... Appendix 28

A.2.1. ADM.MOD ..... Appendix 29
A.2.2. Exceptions.MOD ..... Appendix 53
A.2.3. Expression.MOD ..... Appendix 55
A.2.4. Interface.MOD ..... Appendix 66
A.2.5. Lex.MOD ..... Appendix 74
A.2.6. Listing.MOD ..... Appendix 81
A.2.7. Location.MOD ..... Appendix 89
A.2.8. M2Assem.MOD ..... Appendix 90
A.2.9. ObjectGenerator.MOD ..... Appendix 93
A.2.10. PseudoOps.MOD ..... Appendix 96
A.2.11. Strings.MOD ..... Appendix 101
A.2.12. Table.MOD ..... Appendix 105
A.2.13. TableExt.MOD ..... Appendix 108
A.2.14. TableTrees.MOD ..... Appendix 112

A.3. Testing Modules ..... Appendix 115

A.3.1. TestLex.MOD ..... Appendix 116
A.3.2. TestStrings.MOD ..... Appendix 118
A.3.3. TestTable.MOD ..... Appendix 120

2. ABSTRACT.

The project documents the historical development of the meta-assembler, the possible types available, principles of operation and methods of construction. Various implementations and suggested methods of implementation are documented.

A sample implementation of a meta-assembler is developed using the principles of Object Oriented Software Construction in the high level language Modula–2. The assembler is written so as to conform to the IEEE Proposed Assembly Language Draft Standard of 1979. The report documents the complete development process from Problem Definition to Implementation and Testing. It is concluded that the implementation developed compares well with previous meta-assemblers in terms of speed, flexibility and the ease with which assembler definitions can be written.

3. INTRODUCTION.

This report details the work carried out for a final year project. This project was undertaken throughout the third

year of the BSc Honours Combined Studies Course at The Hatfield Polytechnic.

The development of meta-assemblers has been in progress since the mid-1950's. Ferguson, in his report on the evolution of the meta-assembly program[26], defines the meta-assembler.

> 'A meta-assembly program is a machine independent assembly program. It is machine independent in the sense that both the assembly language which it is to accept (the source language) and the machine code which it is to generate (the object language) are supplied as part of the program to be assembled. That is, both the source language and the object language are input parameters to the meta-assembler.'

This was the first article which clearly defines the concepts of meta-assemblers. Since then, many projects have been carried out in this field. They have presented various solutions to the problem, many of which have compromised on the generality of the meta-assembler. Other terms have been introduced which describe these assemblers. They include Universal Cross-Assembler, Table-Driven Assembler and Program Meta-Assembler. For sake of simplicity, if not for absolute correctness, I will encompass all these names under the heading of meta-assembler and meta-assembly pointing out, where relevant, which class of meta-assembler is being referred to.

There were two primary objectives in conducting this project. These were:

1. To document the development of meta-assemblers from their earliest forms to present day implementations. This includes details about construction techniques, classifications, implementation methods and principles of operation.

2. To attempt a sample implementation of a meta-assembler, to gain a grasp of the problems and possible solutions to the meta-assembly concept.

The project report structure therefore reflects these two objectives. Section 4 is concerned with meta-assemblers and Section 5 is concerned with the sample implementation. Section 6 concludes the work carried out during the project and speculates on the future for meta-assemblers.

4. META-ASSEMBLERS.

4.1. The Need for Meta-Assemblers.

For every new processor introduced, a corresponding assembly language is also introduced. This assembly language provides a means both to program the new processor and also to provide a programmers model of the processor (a hardware description). This assembly language may be a derivation of an earlier assembly language (such as the assembly language for the Motorola MC680x0 series of processors), although it is more likely to be a completely new assembly language. To program this new processor, therefore, requires an assembler. This is a program that translates the mnemonic, human-readable assembly language into a binary machine code, which can be subsequently executed by the processor. For nearly every new processor introduced a new assembler must therefore be coded.

An assembler is quite a complicated program, and consequently can take man-months to produce. In the early days of computing, this was not considered a problem because of the relatively small number of processors in use. However, this situation changed dramatically with the introduction of the microcomputer. A proliferation of microprocessors emerged, each one requiring an assembler.

This proliferation of new processors led researchers to explore the concept of the meta-assembler. This is essentially a software tool which aids the construction of new assemblers, by automating much of the design and implementation.

The roots of meta-assemblers stretch back, however, to before the introduction of the microcomputer. Ferguson[26] mentions in his report on meta-assemblers that many of the ideas behind them were being considered back in the mid-1950's, although at that time meta-assemblers were being used for a different purpose. High-level languages had not yet been invented, so the highest generation of language processor was the meta-assembler. This provided a higher level of programming than the basic assembly language by incorporating concepts such as macros and procedures to define the assembly language specific operations in an independent form. These meta-assemblers developed from the concepts of automatic programming and universal coding. Ferguson discusses one of the first meta-assembler implementations, called UTMOST, which was introduced in 1962. Several other implementations are also mentioned, including FAP (Fortran Assembly Program), SLEUTH, META-SYMBOL and METAPLAN. These, together with other more recent implementations are discussed in Section 4.7.

4.2. Features of Meta-Assemblers.

Skordalakis, in his article describing meta-assemblers[24], puts forward five features of which an ideal meta-assembler should posses. These are:

1. Portability. The programming language used to code the meta-assembler should be one which is easily transported between many different computer systems. Traditionally, third generation high level languages have met these criterion best because (a) they are relatively machine independent and (b) they are available on a wide range of systems. The usefulness of a meta-assembler is obviously enhanced immensely if it can be used on a number of computer systems. Portability also allows the possibility suggested by Ferguson[26] that advantages can be made by assembling for a processor using the same processor.

2. Ease of Use. This is referring to the ease with which a new assembler can be developed. Meta-assemblers which require the assembly language definition to be supplied in the form of a program are generally regarded as more

powerful but also more difficult to use[15]. Obviously, the man-effort required to produce an assembler using a meta-assembler must be significantly less than the effort required to produce a dedicated assembler for the same processor. A number of alternative methods of expressing the assembly language definition have been devised (see Section 4.7, Implementations).

3. Power. This criterion refers to the power of the assemblers which can be produced by a meta-assembler. The power of an assembler is difficult to measure, but a number of features are generally regarded to constitute a powerful assembler (see Section 4.4, Assembly Language).

4. Non-restrictive. The possible range of syntactic and semantic structures which can be included in assembly language programs that can be assembled by the assemblers that can be implemented with a meta-assembler should ideally be unlimited. Program based meta-assemblers are generally the most non-restrictive because of the flexible method by which an assembly language can be defined. More restrictive is the Universal Cross Assembler or Table-Driven Assembler because of the limited variations which can be expressed in tabular form.

5. Generality. This feature is really related to the last. Skordalakis in his report also considers microprogram assemblers which are used to assemble programs which form the microcode of a processor. By generality, he means that an assembler can handle both conventional level assembly programs and microcode assembly programs (there are great differences in the format of the object code produced).

Skordalakis mentions that he had, at that time (1983), not come across any meta-assembler which possessed all these ideal qualities.

4.3. Meta-Assembler Classification.

The classification of meta-assemblers was detailed by Skordalakis in his report on meta-assemblers[24]. Figure 1 shows the classification of the more general term *assembler,* which incorporates both dedicated assemblers and meta-assemblers. A *dedicated-assembler* is the traditional view of an assembler, i.e., one which has been coded specifically to assemble one assembly language and cannot easily be adapted to assemble another. The principles of operation of the dedicated assembler are shown in figure 2.

Meta-assemblers can be divided into a number of categories. The first selection depends on the time at which the meta-assembler becomes dedicated to a specific assembly language. A *compile-time meta-assembler*, as its name suggests, becomes dedicated at compile-time. The assembly language specifics must be contained in a manner

which is incorporated into a meta-assembler shell, which contains the code for the assembly-language-independent components of the meta-assembler. The definition of the assembly-language-specifics might be contained in source code statements, tables or a mixture of both.

Meta-assemblers which are dedicated at run-time, like any system which incorporates late-binding of components, tend to be more complicated to implement[9]. There are two types of *run-time dedicated meta-assembler*, the generative type and the adaptive type. The principles of operation of these two types are shown in figure 3 and figure 4. In a *generative meta-assembler* the assembler description is used to produce a dedicated assembler based on that description. A number of dedicated assemblers can therefore be produced by one meta-assembler by writing many assembler descriptions. The *adaptive meta-assembler*, on the other hand, takes the assembler description file and adapts itself to mimic the corresponding dedicated assembler. Thus, although many assembler descriptions are required there is only one assembler program.

The adaptive assembler can furthermore be broken down into two types, depending on the method used to describe the assembler to the meta-assembler. In a *Ferguson-Type assembler* the assembler description takes the form of a macro language. Where this description takes another form it can be classed as a *non-Ferguson assembler*. This distinction is made because in Fergusons' report[26] he uses the term meta-assembler solely to describe Ferguson-Type meta-assemblers.

4.4. Assembly Language.

It is now appropriate, before going onto meta-assembler construction, to discuss the properties of assembly language and object code formats. The construction of these obviously has a direct influence on the way in which a meta-assembler must operate to turn one into another.

4.4.1. Assembly Language in Context.

Assembly language is just one type of language in a sequence which have developed in parallel to the development of computers. The characteristics of these languages can be used to summarise the type of language by putting it into a group according to the relationship between the language, the problem to be solved and the machine on which it is to be solved. These levels of languages are summarised below;

The lowest level language, microprogramming, is not usually used to present problems to a microcomputer but rather to implement the computer itself. As such, they are specialised and not of general interest.

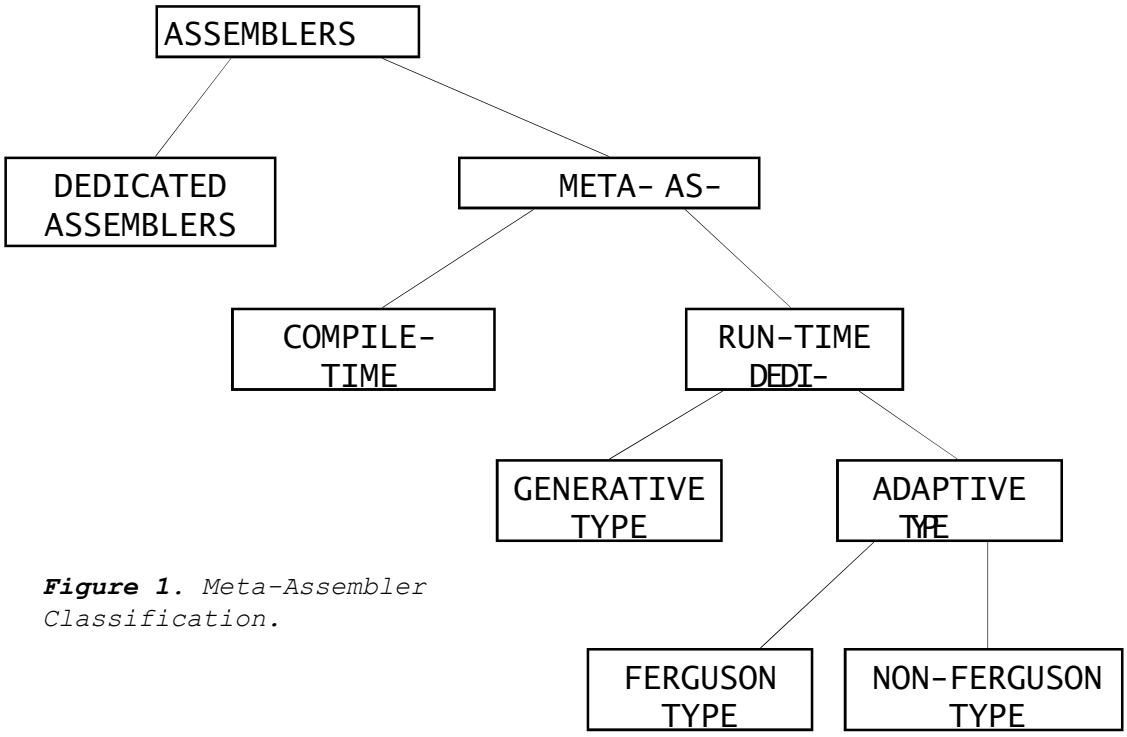


Figure 1. *Meta-Assembler Classification.*

Binary machine languages contain the actual bit patterns for each operation and data element in the problem to be solved. They are directly executable by microcomputers, but prove very difficult for humans to comprehend. For the earliest computers, this was the way in which they were programmed, but the development of the assembler provided a higher level language to specify the problem to be solved, the basic assembly language.

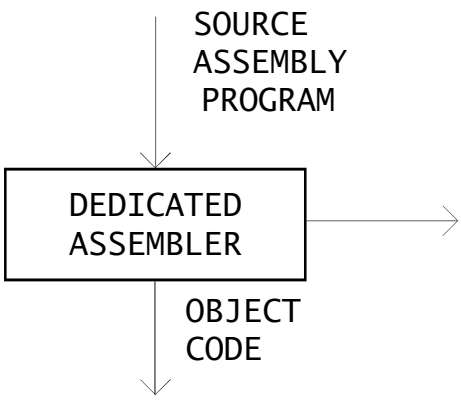


Figure 2. *Dedicated assembler operation.*

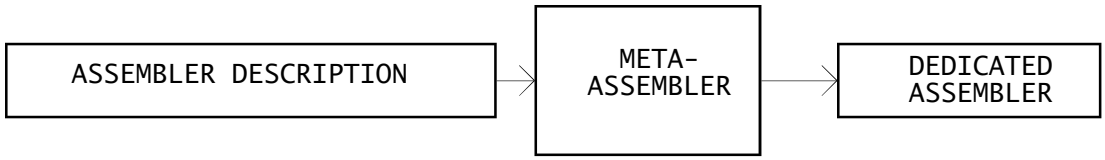


Figure 3. *Generic Meta-Assembler Operation*

The assembly languages (basic and macro) presents the programmer with a human-readable form of binary machine languages. It does this by assigning mnemonic representations to binary commands and decimal/ hexadecimal equivalents of binary numbers. Because of the close relationship between assembly languages and machine languages, most microcomputers have their own specific assembly language. This is used to describe a problem and its solution in terms of the specific resources that a computer is able to offer. Macro-based languages are more powerful in that they allow many basic assembly language instructions to be grouped and manipulated as single entities.

Procedural-Oriented (high level) programming languages abstract from the implementation details and allow the specification of the solution to a problem to be made without regard to the computer on which it is operating. This is an ideal description to which languages such as Pascal, Modula-2 and ADA, amongst others, achieve with a high level of success.

Very high level functional programming languages (such as PROLOG) allow a problem to be specified without regard to its solution. The microcomputer is responsible for providing the means and solution to the problem. There are currently few languages which meet this specification.

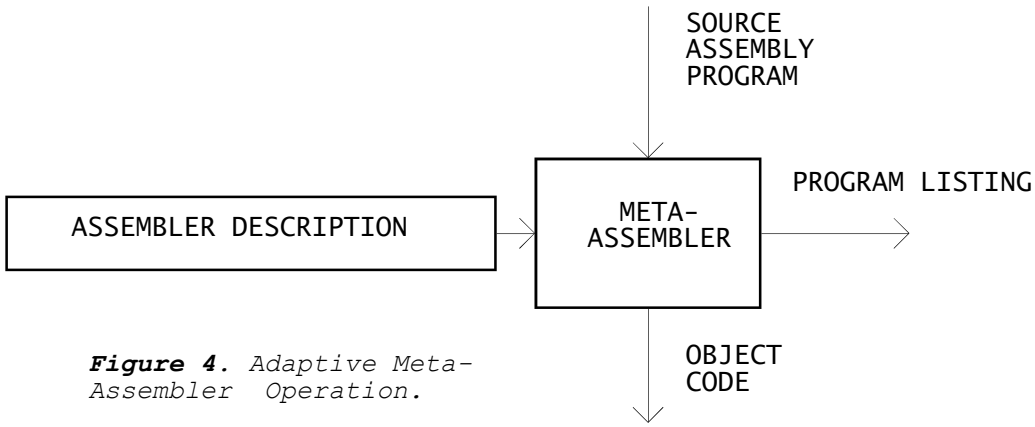


Figure 4. *Adaptive Meta-Assembler Operation.*

4.4.2. Assembly Language Structure.

As was stated before, assembly languages relate directly in the way in which they are structured to machine languages and thus to individual processors. Each processor, therefore, requires an assembly language. As each processor has a different structure and set of facilities, the assembly languages are consequently different. Because of the historical development of assembly languages, however, their overall structure is generally very similar.

An assembly language program for virtually every processor can be broken down into two main structures. The first is an overall structure which contains a mix of assembly language dependant and independent features. The second is the most general structure the assembly language line or statement. The statement can have one or more fields which are separated either by whitespace or specially designated characters. One or more of these lines form an assembly program. The general structure of one of these lines is:

[LABEL] OPERATION OPERANDS [COMMENT]

where optional fields are indicated with square brackets []. The optional label field is a symbolic name chosen by the programmer to represent the current memory location of a command. It could be used to identify the target of a branch instruction, the location of a data container or a procedure name. The label usually starts with an alphabetic character and can be followed by alphabetic characters, numeric characters or special characters such as the underscore (_). The label is separated from the rest of the statement by either whitespace or a colon. The significant length of the label varies between assembler implementations but is typically 8 to 30 characters long.

Language Level	No of language state-ments per machine language instruction	Genera-tion
Very high level	One to very many	4-5
Procedural level	One to many	3
Macro-assembly level	One to some	2
Basic assembly level	One to one	1
Binary machine level	-	

The operation code is generally a short mnemonic name describing either a valid machine operation or an assembler directive. A machine operation describes a valid instruction to be executed by the target processor. An assembler directive is an instruction which tells the assembler to modify its operation in some way. Two types of assembler directive can be identified:

Assembler Independent - These are directives which are included in virtually all dedicated assemblers. The directives, although having varying formats, perform similar or identical functions. These include:

* Output listing control.
* Expression equates.
* Data block reservation/assignment.
* Assignment of code origin.
* End of input file marker.
* Expression evaluation default base.

Assembler Dependant - These are directives which refer to assembler features which match requirements in the object code to be produced. The object code being produced in turn depends on both the target processor and the operating system being used. Generally, assembler dependent directives are concerned with the correct structuring of code into blocks or procedures. The variability of assembler dependent directives is not limited to target processors, therefore, but also to different target operating systems.

Assembler directives are also known as *pseudo-ops* and *pseudo-instructions*.

The operands field contains one or more designations for data to be operated on, usually separated by commas. The maximum number of operands is determined by the target processor, except when dealing with assembler directives. Each operand may refer to one of three things,

1. A constant value.
2. A processor register.
3. A label.

Depending on the requirements of the machine instruction, one or more of these form an operand. Operands can also be formed by an expression consisting of one or more of the above (this depends on the capabilities of the assembler).

The last field, which is optional, is the comment. They are used by the programmer to imbed descriptions of the operation or meaning of the assembly language to the human reader and are consequently ignored by the assembler. Comments are usually separated from operands with either whitespace, a semi-colon, or both and they can appear on their own on an instruction line. Statement lines can also be blank.

4.5. Machine Language Formats.

Each assembly language statement in the input file to an assembler, apart from assembler directives, is translated into a machine language instruction. The format of machine language instructions is determined by the target processor on which it will execute. The format is chosen by the processor chip designers and is based on a design criteria of their choosing. A number of important decisions are made which affect the format of the machine language[1].

4.5.1 Design Criteria in Machine Instructions.

The most important decision to be made is the possible range of instruction lengths to be used. Shorter instruction lengths (in number of bits) are desirable because they lead to both memory space and speed improvements.

A program consisting of *n* 16-bit instructions takes up half the memory space as *n* 32-bit instructions. Improvements in speed are made because the transfer rate from memory to processor is fixed by technological constraints. If the transfer rate of a memory device is *t* bits per second and the average machine instruction is *r* bits long then the memory device will, on average, deliver *t/r* instructions per second. The rate at which instructions can be executed by the processor is directly proportional, therefore, to the instruction length. Shorter instructions can be executed quicker than longer ones. As CPU's become quicker the amount of time required to fetch an instruction from memory becomes more significant.

There are a number of ways to overcome this problem (termed the 'Von-neuman bottleneck' after the inventor of this type of computer architecture) one of which is to reduce the number of bits required to describe an instruction.

The length of instructions should therefore be kept as short as possible. This factor must be offset by the number of instructions required. An n-bit word can only support 2^n different instructions. This total must not only include all possible opcodes but also all possible memory locations to be addressed in the operands field of the assembly language instruction (see Figure 5).

The compromise made by many processor designers is to have more than one possible instruction length. The ideal processor would have very short instruction lengths for the most frequently used instructions with longer instructions being required less frequently. This ideal is limited in practice by the complexity of the most frequently used instructions.

Machine instructions including addresses are at once limited in length in that a minimum amount of space is required for the address in the instruction. Many processors overcome this problem by providing the same instruction with a number of possible address lengths. The assembler can therefore choose the minimum address length which is sufficiently large enough to hold the address required.

4.5.2. Layout of Machine Languages.

The layout of machine languages is, as stated above, a product of the processor designers' design criteria. There are three general principles which machine languages can exhibit[21].

Regularity.

This principle refers to the way in which information such as number of operands, data and address lengths, is encoded in the instruction. A high degree of regularity implies that the method for encoding information is consistent across the possible range of instructions.

Orthogonality.

A high level of orthogonality implies that any machine language instruction can be consistently broken down into a number of independent fields.

An example of an orthogonal machine language is that of the IBM 370 processor in which the first 8 bits of any instruction are always the encoded opcode. Data and addresses then follow in a consistent manner and give rise to only 5 possible instruction formats.

Composability.

This refers to the above two principles. It should be possible to compose orthogonal, regular notions in arbitrary ways.

It should be noted that while all these principles lead to a more intuitive instruction layout and simple assemblers, they give rise to longer instructions and therefore a reduction in efficiency both in terms of speed and memory usage (see the previous section).

The design criteria used determines the degree to which any particular machine language meets these ideals. The best way to identify possible machine language formats is by example.

The Motorola MC68000 Machine Language.

Figure 6 shows the method of encoding used for the first machine language word (16 bits) of the MC68000. The reason for there being so many different formats is because the MC68000 has a large number of possible instructions compared with the size of the word and there are therefore few bit patterns unused. The MC68000 designers, rather than opt for the principle of orthogonality, decided to allow more frequently used instructions more opcode space and therefore more flexibility on the number of variants possible. The MOVE instruction, which is the most important instruction in any machine language, is encoded using instruction format 1 (see figure 6). This instruction allows data transfers to take place between a number of possible locations including memory to memory, register to memory, memory to register and register to register. Immediate data can be transferred to both memory or registers. Almost 1/4 of the total possible bit combinations is taken with this one instruction alone.

With this fact in mind, it was clearly impossible to provide two 6-bit operands for all the other instructions. Most of the other instructions, therefore, have one general purpose 6-bit operand and a 3-bit register operand. So, for example, with the ADD instruction (format 2) it is impossible to add an operand located in memory to another in memory. One of the operands must be a register.

The squeeze of instructions into the 16 bits available is evident by the peculiar method with which some instructions are encoded. For example, the MOD field in instructions with format 2 distinguishes between operations on byte (8-bit), word (16-bit) and long-word (32-bit) operands as well as determining whether the operand refers to the source or destination of the instruction. As this only needs 3 X 2 = 6 possibilities and the 3-bit MOD field allows for 8 possibilities, the other two MOD values are used to indicate completely different instructions.

Another good example of the full use of the first 16-bits of any instruction is the similarity of the four instructions S_{cc}, DB_{cc}, ADDQ and SUBQ (see figure 7).

All four instructions have the same bit pattern in the upper (12-15) four bits. The latter two instructions are distinguished from the upper two by the presence of the 3 in the SIZE field. This is an illegal size in the ADDQ and SUBQ instructions. The separation of the S_{cc} and DB_{cc} instructions is even more difficult as potentially each can have the same 16-bit combinations. This problem is overcome by making MODE = 1 in the S_{cc} instruction illegal, thus giving the DB_{cc} instruction an allowable bit pattern.

It can be seen that the MC68000 instruction set, while providing some degree of orthogonality, is definitely not regular as special cases of some fields give rise to completely different instructions. It follows that the instruction set is not very composible either. All these factors, while working to the advantage of more instructions in the same instruction space, inevitably lead to more complicated assemblers and compilers. Special cases have to be taken into account in a fully implemented assembler, although relatively simple assemblers can be written if a number of the more complexly encoded instructions are ignored[1].

The Zilog Z80 Machine Language.

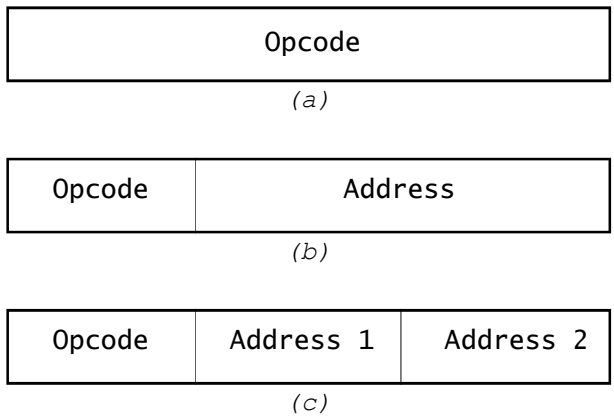


Figure 5. *Three typical instruction formats: (a) zero-address instruction. (b) one-address instruction. (c) two-address instruction.*

The format of the Z80 Machine Language is neither regular, othorgonal or composible. This is mainly due to the decision taken by the Z80 designers that it should be bit-compatible with the earlier 8080 processor which in turn was semi-compatible with the 8008 processor (which was an 8-bit version of the 4004!). This meant that all the new Z80-specific instructions had to be encoded using the spare opcode patterns of the 8080 processor, because the Z80 had to be able to run unmodified 8080 programs. The instruction formats which resulted are obviously more complex than they would have been if they had been designed from scratch. In total there are 26 possible instruction formats.

Unlike the MC68000, the length of Z80 instructions is not fixed by the length of data being operated on. Instead, the length of an instruction is mainly determined by its frequency of use. Thus frequently used instructions (such as a data move from one register to another) are encoded using only 8 bits whereas less frequently used instructions can be up to 32 bits long. The advantage of designing a machine instruction set this way was discussed earlier in the previous section, mainly that memory usage and instruction transfer times from memory to processor are reduced. The main disadvantage is that instructions generally take longer to decode.

Assemblers and compilers for the Zilog Z80 processor are necessarily even more complex than those for the MC68000. The special cases which must be considered include all the new Z80-specific instructions.

4.6. The Assembly Process.

The purpose of an assembler is to translate a source program in assembly language into an object program in machine language (see figure 8). The method by which this translation occurs is very data-oriented in that what is essentially happening is the transformation of data. The assembly process can therefore be broken down into a number of *phases*, each one of which performs transformation on some input to provide output. Comparisons can be made between the phases of an assembler and the phases of a compiler. An assembler is, in effect, a simple compiler.

Watson[14] identifies two broad sub-tasks which must be performed by a compiler.

1. The analysis of the source program.
2. The synthesis of the object program.

In the case of an assembler, the source program is written in the assembly language of the target processor and the object program is written in the machine language of the target processor.

The analysis task can be broken down into three phases:

1. Lexical Analysis.
2. Syntax Analysis.
3. Semantic Analysis.

The synthesis task is usually considered as a single *code-generation* phase (see figure 9).

The interface between the phases of an assembler can vary between different implementations. Each phase could, for instance, be a separate program transforming an input file into an output file. Such an assembler would be a four-pass assembler since four passes are made over various versions of the source program. Four-pass assemblers are not very common, however, because they are extremely inefficient. A much more efficient way of providing the interface between phases is via data structures containing tokens. A token is a basic syntactic component of the input language of which four were identified in section 4.4.2. These are:

1. Label.
2. Operation.
3. Operand(s).
4. Comment.

These could be passed either individually or in groups between the phases of an assembler. The most convenient grouping is that of an assembly language statement. In this way, two-pass and one-pass assemblers are possible which avoid the need to read and write several versions of the source program.

The decomposition of an assembler into distinct phases has a number of advantages, not least of which is that it makes the task of writing what is a large program easier. The other major advantage of decomposition in this way is that all but the code generation phase can be made machine language independent. Thus it can be seen that this introduces some of the properties of meta-assemblers.

4.6.1. Lexical Analysis.

The task of the lexical analyser is to read characters from the source program and group them into basic syntactic components which can be passed onto the syntax analyser as tokens.

In an assembler this task becomes fairly trivial because of the small number of syntactic components used and the

rigid structure which is imposed on them. The lexical analyser takes the free-format source program and extracts tokens by ignoring all whitespace (space and tab characters) and comments (these are not needed by subsequent phases).

The output from the lexical analyser could take a number of forms. It might produce pointers to the start of each of the tokens in the source program. For example, the statement

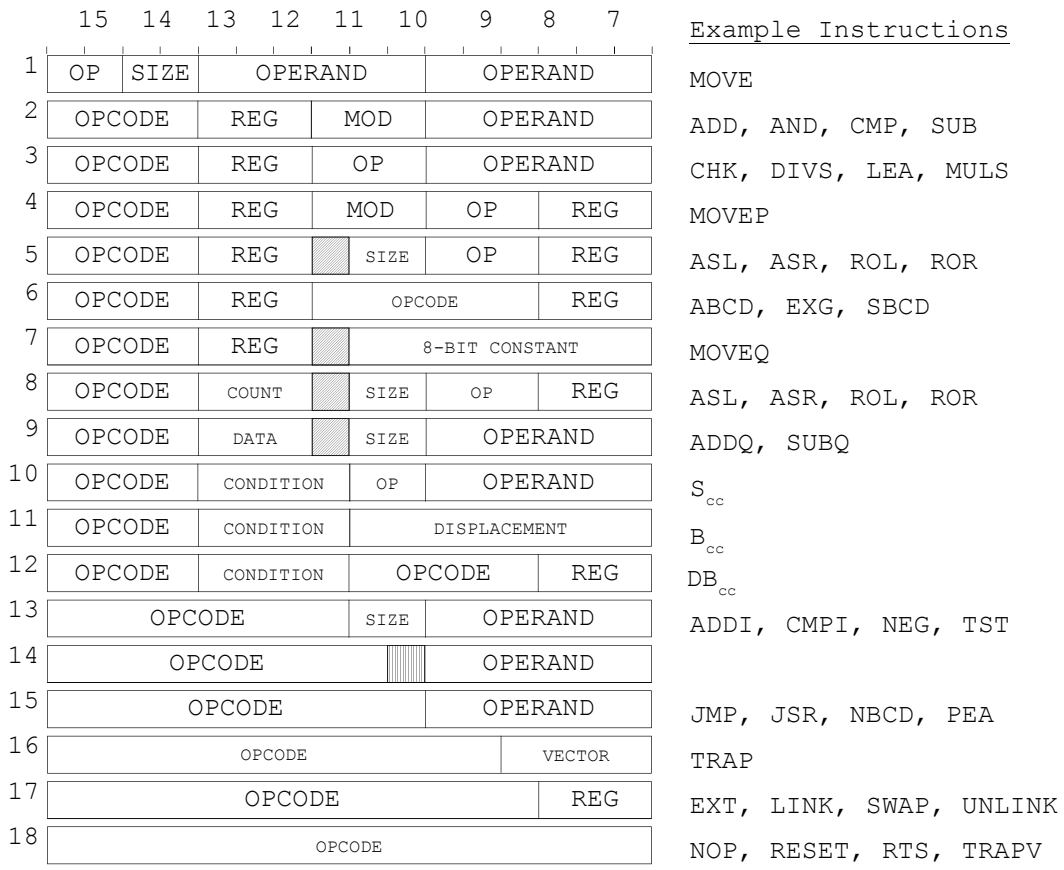


Figure 6. *Instruction formats used on the MC68000 (first word only).*

```
Start:        MOVE         D0, D3       ;comment
^             ^            ^
Label         Operation    Operands
```

would produce three pointers to the start of the label, operation and operand fields. These could be CARDINAL numbers representing the position in the source line or could be POINTERs TO CHAR.

Another method of expressing the output of the lexical analyser could be to fill a data structure with copies of the fields in the source program line. Given a declaration of a RECORD structure

```
TYPE
  String     = ARRAY [1..30] OF CHAR;
```

ADDQ	0	1	0	1	DATA	0	SIZE		MODE			REG
SUBQ	0	1	0	1	DATA	1	SIZE		MODE			REG
S_{cc}	0	1	0	1	CONDITION		1	1	MODE			REG
DB_{cc}	0	1	0	1	CONDITION		1	1	0	0	1	REG

Figure 7. *Four MC68000 Instructions.*

```
  LexOutput   = RECORD
                  Label     : String;
                  Operation : String;
                  Operands  : String
                END;

VAR
  CurrentLine : LexOutput;
```

the following assignments would be made by the lexical analyser:

```
CurrentLine.Label     := "Start";
CurrentLine.Operation := "MOVE";
CurrentLine.Operands  := "D0, D3";
```

The advantage of the first method is that duplication of data is avoided but subsequent phases in the assembly process will have to determine where tokens end. The second method provides tokens as separate entities but sacrifices memory to do so.

Errors produced by the lexical analyser relate to the way in which tokens are formed. They might include:

* Label starts with a non-alphabetic character.
* Operation contains non-alphabetic characters.
* Unexpected end of line.

4.6.2. Syntax Analysis.

The syntax analyser must decide how to group tokens given to it by the lexical analyser. With assembly languages, the tasks of the lexical analyser and the syntax analyser are very difficult to distinguish because tokens come in only one group, the statement. Whereas the output of the syntax analysis phase of a compiler would probably be a parse tree, this phase in an assembler essentially produces the same output as the lexical analysis phase. However, although the data may not change from input to output an important operation still takes place. This is the checking of the source code for syntax errors. These are errors in the structure of the source program. Examples could be

* Unknown operations.
* Illegal numbers of operands.

4.6.3. Semantic Analysis.

In a compiler, the semantic analysis phase determines the meaning of the source program. It translates the groups of tokens produced by the syntax analyser in the form of a parse tree into a linear sequence of instructions in an intermediate language.

In an assembler, the semantic analyser performs a number of tasks.

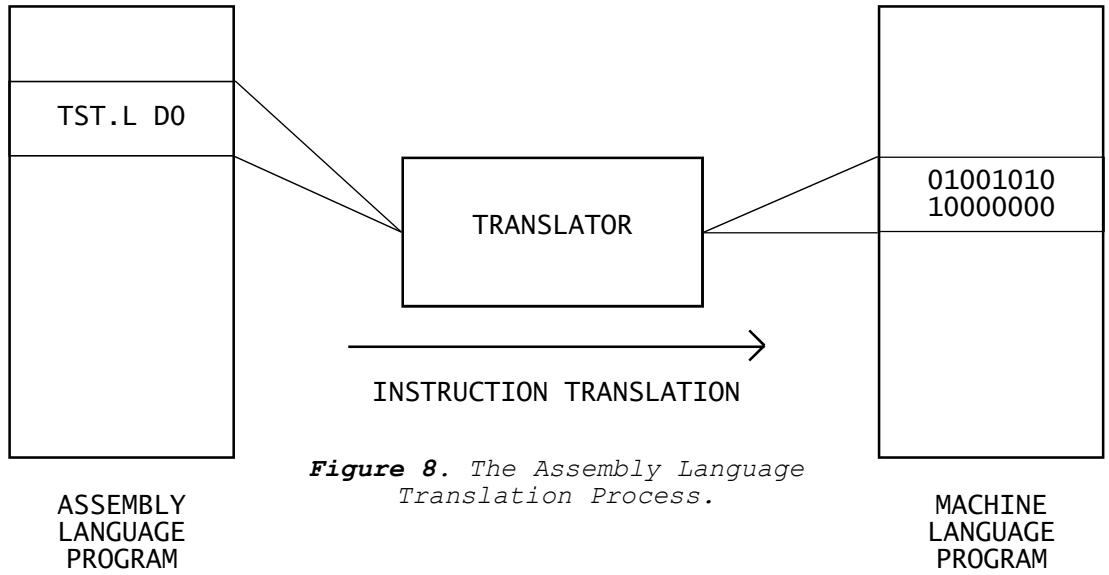


Figure 8. *The Assembly Language Translation Process.*

1. It must extract information about the operation such as the type (either assembler directive or machine instruction) and the valid number and type of operands.

2. Operands must be decoded to determine their addressing mode. A cross-check is made between the addressing modes of the operands and the addressing modes which are legal to the operation being performed.

3. Operands must be evaluated. This involves expression evaluation where arguments could be labels or constants in a number of different bases, and where operations could be arithmetic or logical.

Errors produced might include:

* Invalid expression.
* The wrong use of labels.
* Bad addressing modes.

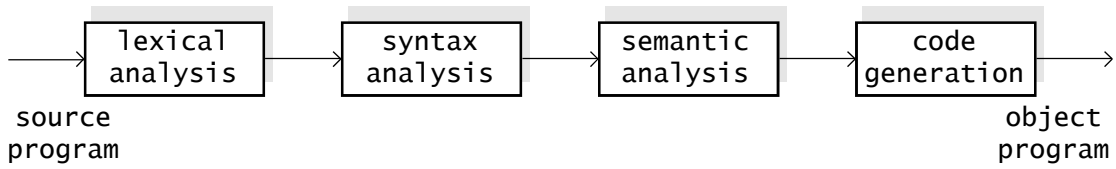


Figure 9. *Phases of the Assembly Process*

The output of the semantic analyser of an assembler could take the form of a more detailed structure similar to the one produced by the lexical analyser, with fields for

1. Operation mask (if appropriate).
2. Operation type (assembler directive or machine instruction).
3. Operand values.
4. Operand types (in terms of addressing modes).

4.6.4. Code Generation.

The final phase of the compilation process is to take the output from the semantic analyser and produce a machine code program which is ready to execute on the target processor.

In a compiler this involves translating the intermediate code generated by the semantic analyser and substituting these with the appropriate sequences of machine language instructions.

In an assembler, the process is simplified in that each of the 'intermediate instructions' produced by the semantic analyser maps directly onto one machine language instruction. Machine instructions are created by taking each element in the data structure produced by the previous phase and slotting them into the correct position. Conversion of operands into formats required by the target processor (e.g., twos complement integers) might also be required. The code generator calls on inbuilt knowledge to determine, for example, where operands should be placed.

When a machine instruction is completely assembled it is entered into the output object code program which imposes some form of structuring. This must also be done in the code generation phase.

Whereas optimisation might take place in the code generator of a compiler, this is a very small part of an assembler. Any optimisations which can be made by the assembler usually only have a small effect on the efficiency of the code produced. Examples of optimisation which are possible are usually restricted to substituting a more efficient version of a machine instruction for another.

4.6.5. A Simple Two-Pass Assembler.

It is now appropriate to give an example of the way in which the phases of an assembler are brought together form

a complete assembler.

The correlation between the *passes* and the *phases* of an assembler is somewhat ambiguous. This is mainly because the development of assemblers has lead to the most efficient implementations which do not necessarily match the idealised model of an assembler.

Most assemblers require two-passes because of the *forward reference problem*. This occurs because of the sequential way in which the assembler scans the source program. Most assemblers use the group of tokens which form a statement in the source program as their method of communicating between the assembly phases. Each phase works on one statement and then passes this onto the next phase. This is done because of the high overhead required to read and write many intermediate files. The forward reference problem occurs when an assembly language statement refers to another later on in the source program. This forward reference usually takes the form of a label which has not yet been defined. Consider the MC68000 program in figure 10.

The assembler will begin by defining the symbol **Start** and inserting it in a symbol table. The symbol table contains the names of symbols together with their values. In the case of the symbol **Start**, its value would be zero because the *Instruction Location Counter* (ILC) is reset to this at the start of the assembly process. The compare instruction is then assembled and its machine code equivalent inserted in the object program. The ILC is incremented by the length of the current instruction. When the assembler analyses the operand in the next instruction it finds that it cannot give it a value because the label **NotEqual** has not yet been defined. The easiest way to satisfactorily resolve this problem is to have all symbols in the source program available with their values before the assembly process begins. This is what gives rise to the first pass on a two pass assembler.

If the problem is regarded conceptually from the point of view expressed in the previous section (i.e., assembly by phases) then the phases of lexical analysis, syntax analysis and semantic analysis are repeated twice during assembly. All the phases are needed during the first pass because a preliminary analysis of the operation and operands in a statement is required in order to determine the instruction length. This is needed in order to increment the ILC by the correct amount. The output from the first pass of the assembler is a symbol table. Logically, this should be in addition to a data structure containing all the assembly language statements in a decoded form (similar to the output of the syntax analyser in section 4.6.2.). However, the overhead in maintaining two files of data is usually considered greater that the extra time necessary to perform all these stages again in the second pass. No intermediate file is generated between passes, therefore, and the second pass re-reads the source file but with the values of all symbols in it defined.

Figure 11 gives a procedural example of what the first pass of a two-pass assembler could look like. Apart from the symbols defined as labels for statements, most assemblers allow the explicit definition of symbols using an

assembler directive. So, for example, the statement

```
BufferSize        EQU   256
```

tells the assembler to store the symbol BufferSize in the symbol table with value 256.

The Instruction Location Counter managed by the assembler can usually be referred to in the source program via a special symbol, a common example being the asterisk character. The value of the ILC can be explicitly modified in the source program by using an origin directive. The statement

```
        ORG     2000
```

sets the value of the ILC to 2000. This allows programs and parts of programs to be located anywhere in memory.

The second pass of the assembler performs the complete set of phases described previously to translate the source program into the object program, with the source program being accompanied by a complete symbol table. Figure 12 gives a similar procedural view of the second pass.

The semantic analyser makes use of an opcode table which contains details about all the machine language instructions available on the target processor. Details include:

1. The mnemonic name of the instruction.
2. The basic length of the instruction (which may be modified by the size of the operands on some processors).
3. A basic mask containing all the bits set by the instruction opcode on its own.
4. The class of instruction. This corresponds to one of the possible formats which the instruction can take (see section 4.5.2.).

When operands have been evaluated with an expression evaluator (which uses the symbol table where necessary) they can be added to the opcode mask by an instruction-format-specific routine selected with the class of opcode indicated above.

4.7. Implementations.

In this section various implementations of meta-assemblers which have been developed are described. This will be done in chronological order so as to give a mini-history. It can be seen that meta-assemblers and their purpose

for being developed has evolved from the early days of computing up to the present.

4.7.1. Early Implementations.

Published reports on early implementations of meta-assemblers are limited to one written by Ferguson in 1966[26]. In this report, titled 'The Evolution of the Meta-Assembly Program', Ferguson details both the origins of meta-assemblers and early implementations. His view of meta-assemblers and later views are significantly different because of the way in which they were being used at that time. They were, in effect, low-level languages being described using high-level language constructs. This allowed direct access to machine-specific operations via a higher level than was currently being provided by assembly languages. The method used to provide this functionality included the many-to-many macro (as opposed to the one-to-many macro common in most macro-assembly languages today) which was referred to as procedures and functions. The consequence of being able to express machine-dependent features in a high-level way was to significantly reduce the problems associated with writing programs for more than one processor. A side-effect of this also made it possible to convert programs written for one processor into a form which could be assembled for another.

A report by the United States Navy in 1954 contained a discussion on many of the ideas involved in meta-assembly. The report was on automatic programming, a concept which together with universal coding provided the basis of the principles behind meta-assemblers. The first program which could truly be called a meta-assembler was UTMOST which was completed in 1962. It allowed the description of the relationship between the assembly and machine languages of a processor to be described in terms of procedures with generalised arithmetic expressions and lists for parameters. An implementation called FAP (FORTRAN Assembly Program) introduced the concepts of a relocatable location counter and the external symbol which are used in both meta-assemblers and dedicated assemblers today. A later implementation of UTMOST called SLEUTH expanded on the procedure concept by the introduction of the function i.e., a value returning procedure. META-SYMBOL, an implementation for the SDS 900 series of computers, added on the SLEUTH implementation by generalising the list concept even further and by the introduction of a new method for encoding symbolic information into binary. This significantly reduced the volume of information required to describe any particular assembler specification. An implementation of a meta-assembler for the Spectra-70 series of computers introduced the many-to-many macro which enabled it to efficiently translate any source program written in METAPLAN (META Programming LANguage).

A book written by D.W.Barron called Assemblers and Loaders[2] which was first published in 1969 has a chapter on meta-assemblers, in which he comments on Ferguson's report. Barron makes the important point that the syntax

of the input programs to all the meta-assemblers described by Ferguson is fixed. So, although these meta-assemblers could be configured to assemble for virtually any machine (at the time) they could not assemble programs written in the assembly language provided by the processor's manufacturer. Later implementations address this problem.

So, with all the meta-assemblers up to 1969 (to my knowledge), the assembly process looked somewhat like figure 13. The main difference, therefore, between these early meta-assemblers and later implementations is that both the assembler specification and the program to be assembled were in the same file. The assembler specification took the form of normal source statements where the operation field is interpreted as a procedure call with the operands field being interpreted as parameters to that procedure. Symbols defined in expressions which made up the operands could take either normal symbolic values or could be names of functions which were described in the source file and were, in turn, evaluated by the meta-assembler. Predefined procedures and functions allowed the construction of the output object file and listing file. This process is described in more detail in section 4.7.5.

4.7.2. CASS-8: A Universal Cross Assembler.

The re-interest which was expressed by software developers in the concept of the meta-assembler in the late 70's came about because of the proliferation of microprocessors. This was the basis for the development of CASS-8, a cross assembler suitable for most of the 8-bit microprocessors[16]. As regard to microprocessors, two main areas led to the need for meta-assemblers.

1. In educational environments, where many processors are being studied, it is prohibitively expensive to buy manufacturer's assemblers and cross-assemblers for every processor. Another factor is the many different quirks and small differences in syntax found between dedicated assemblers, which throw students into some degree of confusion when several dedicated assembly languages must be learn in a short period of time.

2. In development environments, where engineers develop systems using many different processors. Once again, cost is a major factor not only in buying many dedicated assemblers but also in retraining staff to use all these assemblers.

The CASS-8 assembler was developed to solve problems in the first environment above. It is a table-driven assembler in that the relationship between the assembly and machine languages of a processor is described as a set of tables. This is input into the meta-assembler in the form of a separate file to the program to be assembled. This assembler specification contains the following information for each machine language instruction.

1. The mnemonic of the instruction.
2. The operation code of the machine language instruction (opcode bit-mask) corresponding to the mnemonic above.
3. The type of operands which are legal for this instruction.
4. The number of machine cycles required for execution of the instruction.
5. A flag indicating whether the instruction gives rise to a branch condition.
6. An optional descriptive comment.

As the second field is only one byte in size, only 8-bit processors can be assembled to. The source language is an adaptation of any manufacturer's assembly language in that the operation code must contain the instruction's concerned register (if applicable) and the addressing mode to be applied. This significantly reduces the amount of lexical analysis required because the addressing mode of any instruction is instantly known when the assembler mnemonic is retrieved from the opcode table. All the assembler dependent features are therefore described using data (as opposed to algorithms). This tends to lead to an easier implementation but also to inflexibility in the variety of processors whose machine and assembly languages can be translated. This is reflected in the fact that only 8-

```
Line Label        Opcode    Operands

1        Start          CMP       #10,D0
2                       BNE       NotEqual
3                       CLR       D0
4                       BRA.S     Continue
5        NotEqual  ADD       #15,D1
6        Continue  RTS
```

Figure 10. *The forward reference problem.*

bit processors are catered for and even then some are not applicable. This is because of the strict structuring imposed on the object code generated, which must be of the form of a 1 byte opcode followed by a 0, 1 or 2 byte operand. Eight bit processors which do not conform to this format of object code include the Signetics 2650 and Zilog Z80.

Figure 14 gives an example of the sort of entries required in an assembler specification file for the Motorola M6800 processor instruction set.

4.7.3. A General Purpose Cross-Assembler for Microprocessors.

This is a similar type of meta-assembler to the one above, developed by S._Fujita and K._Yoshida of the

Yokoshuka Electrical Communication Laboratory at around the same time[17]. It is an implementation which is table driven but the assembler specification is in the form of a program which is processed to produce a set of tables. A MASP (Meta-language Assembler SPecification) is converted into a MAST (Meta-language Specification Table). A general purpose assembly program then uses different MASTs to assemble programs for different microprocessors. The principle is shown in figure 15. In this case, both the source language and the corresponding machine language must be specified in the MASP. This means that the general purpose cross assembler can mimic manufacturer's dedicated assemblers. MASP is designed to enable the description of delimiters, mnemonic opcodes, number of operands, expression types of operands and the determination of evaluated value of operands. The assembler directives are kept predefined (assembler-independent) to improve the efficiency of the assembly process.

The report on the meta-assembler concludes by describing the success with which different microprocessors could be described in MASP. Out of 28 processors tested, 26 could be described in some way and 22 (see figure 16) could be described to produce an exact replica of the manufacturer's dedicated assembler (excluding assembler directives). The shortness of MASP descriptions reflect the speed at which they can be produced. Approximately 1/100 of the man/hours is required and 1/60 of the source lines (compared with the development of a dedicated assembler for the same processor). Figure 17 shows the length of MASP files for various processors.

4.7.4. UOPCA - A Table Driven UCA Written in BASIC.

A report by J. Robert Heath and Shailesh M. Patel published in 1981 details their work on the development of a universal cross-assembler in BASIC[19]. The concepts behind the structure of the assembler are very similar to those of Fujita and Yoshida[17]. Being table-driven, restrictions on the source and object syntax are proportional to the complexity of the assembler description tables employed. The assembler differs from the other implementations in that it takes only one pass to assemble any source program (hence UOPCA - Universal One-Pass Cross-Assembler). When the UOPCA encounters a forward reference the instruction is flagged in a temporary file. When the single pass of the source program has been completed the temporary file is read and previously unknown references are now known and can be successfully assembled and written to the object file.

The implementation which was written was an experimental version of the full UOPCA design. As such, error checking, the number of assembler description files and the ability to handle complex machine instruction formats (especially on 16-bit processors) was compromised. The format of the opcode tables also places less obvious restrictions, such as the maximum addressable memory space of only 65536 (64_Kbytes). The structure of the tables rely heavily on the orthogonality of target machine languages. Figure 18 shows the format of the assembler description file. It can be seen that restrictions are also made by the file structure, such as the number of register

```
PROCEDURE PassOne;
(* This procedure is an outline of pass one of a simple assembler. *)

  CONST
    size = 8;
    EndStatement = 99;

  VAR
    ILC, class, length, value: INTEGER;
    MoreInput                : BOOLEAN;
    literal, symbol, opcode  : ARRAY [1..size] OF CHAR;
    line                     : ARRAY [1..80] OF CHAR;

  BEGIN
    ILC := 0;                           (* Initialize Instruction Location Counter *)
    MoreInput := TRUE;                  (* Set to FALSE at END directive or EOF    *)
    InitializeTables;                   (* Call a procedure to set up tables       *)

  WHILE MoreInput DO                    (* Loop executed once per line             *)
    ReadNextStatement(line);            (* Go get some input                       *)
    SaveLineForPassTwo(line);           (* Save the line                           *)

    IF LineIsNotComment(line) THEN      (* Is it a comment?                        *)
      CheckForSymbol(line, symbol);     (* Is there a symbol?                      *)
      IF symbol[1] <> '' THEN           (* If column 1 blank, no symbol            *)
        EnterNewSymbol(symbol, ILC)
      END;
      LookForLiteral(line, literal);    (* Literal present?                        *)
      IF literal[1] <> '' THEN          (* Blank means no literal present          *)
        EnterLiteral(literal)
      END;

      (* Now determine the opcode class. -1 used to signal illegal opcode. *)
      ExtractOpcode(line, opcode);
      SearchOpcodeTable(opcode, class, value);
      IF class < 0 THEN
        TryPseudoInstr(opcode, class, value)
      END;
      length := 0;                      (* Compute instruction length              *)
      IF class < 0 THEN
        IllegalOpcode
      END;
      CASE class OF
        0: length := type0(line) |      (* Compute instruction length              *)
        1: length := type1(line)        (* ditto                                   *)

        (* Other cases here *)
      END;

      ILC := ILC + length;
      IF class = EndStatement THEN
        MoreInput := FALSE;
        RewindPassTwoInput;
        SortLiteralTable;
        RemoveRedundantLiterals
      END
    END
  END
END PassOne;
```

Figure 11. *Pass one of a simple assembler.*

equivalent values (a maximum of 19). This also assumes that the equivalent values for registers do not change between machine language instructions. The format of the object code produced is similar to that of the previous implementation described in that non-orthogonal processors such as the Zilog Z80 cannot be handled satisfactorily. Opcode tables have, however, been written for the Intel 8080 and the Motorola M6800.

This report differs from all others I have come across in that a complete listing of the experimental version of the UOPCA is included, as well as an example assembler specification file for the Intel 8080. Heath and Patel assume that for some processors modification of the assembler source program will be necessary (although not extensive).

4.7.5. XMETA - A High Level Language Based System for Cross-Assembler Definition.

XMETA is a fundamentally different meta-assembler from those discussed so far. The assembler specification takes the form of a high-level language program. This approach addresses the problem of table-driven assemblers which is inflexibility in the assembler specification. The price paid for a more flexible system is in assembler complexity and the ease with which new assembler specifications can be written.

XMETA is extensively documented by its authors in two reports:

1. A High Level Language Based System for Cross-Assembler Definition[23].
2. Experiences with the use of a Universal Cross-Assembler based on the IEEE Proposed Standard[28].

The first report deals with the structure and operation of XMETA, while the second concentrates more on its ability to assemble programs written using the IEEE Assembly Language Standard (more details of which can be found in section 5.3.3.).

The author of the first report and one of the designers of XMETA, M. Anacona, goes into some depths about the possible methods which can be used to obtain a cross-assembler system (other than a table-driven approach). Among these are:

1. The use of Translator Writing Systems (TWSs).
2. The use of High-Level Languages (HLLs).
3. The use of Meta-Assemblers (Ferguson's definition).

Each of these possible approaches provide solutions to parts of the problem, but none provide a completely

satisfactory answer.

Although Translator Writing Systems provide a good method of solving the problem, they are generally large and require a medium-to-large sized host computer. They are effectively an over-kill to the problem (being more in the realm of compilers).

High-Level Languages, such as Pascal, can be used to write very efficient implementations of dedicated

```
PROCEDURE PassTwo;
(* This procedure is an outline of pass two of a simple assembler. *)

  CONST
    size = 8;
    EndStatement = 99;

  VAR
    ILC, class, length, value: INTEGER;
    MoreInput                : BOOLEAN;
    literal, symbol, opcode  : ARRAY [1..size] OF CHAR;
    line                     : ARRAY [1..80] OF CHAR;

  BEGIN
    ILC := 0;                            (* Initialize Instruction Location Counter *)
    MoreInput := TRUE;                   (* Set to FALSE at END directive or EOF    *)

  WHILE MoreInput DO
    GetNextStatement(line);              (* Get input saved by pass one             *)
    IF LineIsNotComment(line) THEN
      ExtractOpcode(line, opcode);
      SearchOpcodeTable(opcode, class, value);
      IF class < 0 THEN
        TryPseudoInstr(opcode, class, value)
      END;
      length := 0;                       (* Compute instruction length              *)
      IF class < 0 THEN
        BadOpcode
      END;
      CASE class OF
        0: length := eval0(line) |
        1: length := eval1(line)

        (* Other cases here *)
      END;

      AssemblePieces(code, class, value, operands);
      OutputCode(code);
      ILC := ILC + length;
      IF class = EndStatement THEN
        MoreInput := FALSE;
        FinishUp
      END
    END
  END
END PassTwo;
```

Figure 12. *Pass two of a simple assembler.*

assemblers. Modifications to a dedicated assembler written in a HLL to make it assemble for another processor usually involve only 20 to 30 percent of the source program. However, the modifications are scattered throughout the text of the source program and a comprehension of the operation of the complete assembler is required. This inevitably leads to an excessive amount of work which can only be performed by the author of the assembler or a programmer who is fully conversant on the operation of the assembler.

The benefits of using HLLs, however, are great. They include a systematic approach to development, greater understanding of cross-assembler structure, extensive debugging aids and lead to an implementation which is highly portable.

Translator Writing Systems also provide a systematic method of development and solve some problems such as syntax analysis and expression evaluation simply and immediately (as these features are already built in). The combination of the two (HLLs and TWSs) has very recently proved an effective solution[32] (see section 4.8.7).

Anacona chooses the concept of meta-assemblers for his implementation, however, because of a number of factors.

1. Encapsulation of most of the details common to all assemblers, thus helping the implementor in several cumbersome problems.
2. Definition of (part of) the syntax and the semantics of the object language by accessing attributes through predefined functions, in an immediate, easy-to-use way.
3. Automatic definition and retrieval of macro libraries, each containing the definition for a particular cross-assembler.
4. Detailed control over code generation.

Previous implementations of meta-assemblers have been macro-based. This lead to a rigidity in the input syntax description, inefficiency, large memory requirements and an almost complete lack of portability between macro-libraries (assembler definitions). The greatest limitation of the implementations from the users point of view, however, was the choice of macros for assembler specifications: they have a cumbersome, non-standardised, unreadable and hard-to-understand syntax. While this was the level of language complexity around the time of Ferguson's report, no evolution had been made to reflect the developments made in high-level language design.

This fact, coupled with attempts made to standardise the syntax of assembly language programs lead to the development of the XMETA system. Its design includes the benefits of the three mentioned methodologies and uses a standardised assembly language syntax. This leads to an effective implementation written in a portable high-level language which means that it can be executed on both mainframes (as a universal cross-assembler) and

microcomputer development systems (as a universal assembler).

XMETA Language.

Figure 19 shows the XMETA assembly process. It can be seen that it is an adaptive, non-Ferguson type meta-assembler (non-Ferguson in the sense that the assembler specification is not macro-based). The assembler specification is written in a Pascal dialect. Each assembly language mnemonic and assembler directive is described as a Pascal procedure with the same name. The procedures describe what action is to be taken to translate the assembly language instruction into a machine language instruction or what action is to be taken when an assembler directive is encountered. Each procedure can make use of built-in procedures and functions to analyse the source program statement and to generate object code, listing lines and error messages where appropriate.

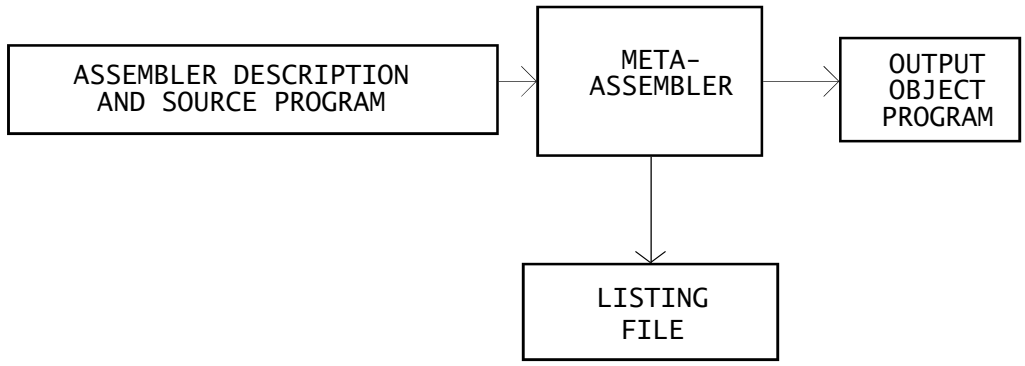


Figure 13. *Early Meta-Assembler Operation*

The language chosen for the first implementation of XMETA was kept to a minimum for the following reasons:

1. To select, at first, just what was considered indispensable, in order to implement extensions only when adequate feedback had been received.
2. So that the resulting implementation was kept as small as possible, easy-to-use and executable on microcomputer systems.

Features of the Pascal dialect used include:

1. Procedures and functions with associated parameters.
2. Integers which can be structured using one-dimensional arrays.

3. Character strings used in output instructions.
4. Numerical values specified in base 2, 8, 10 or 16.
5. *if*, *while* and *case* statements with structured goto's being performed using the *exit* statement.
6. Recursive procedure definition and calls.
7. A 'main program' which is executed at the start of the assembly process and allows the definition and initialisation of assembly variables.
8. Block structure and scope rules in block nesting.

Figure 20 shows the XMETA predefined functions which allow access to the assembly language source statement, which must conform to the IEEE standard[25]. The function **Symbol** returns a character string corresponding to the identifier requested, while all other functions return integer values. **Arg**, **Xf1** and **Xf2** return the results of expression evaluation while **Pref1**-**Pref3** and **Post** return an integer representing the ASCII value of the single prefix/postfix character. A predefined value of **nul** is returned when the requested syntactic item does not exist.

Figure 21 gives an example of the values produced by these functions when applied to a typical IEEE-compatible assembly language statement. Because the symbol **BETA** has not yet been defined, **Arg**(1) returns an arbitrary value and **Type**(1) returns a flag indicating that this is a forward reference. The evaluation is deferred to a later fix-up routine when the value of **BETA** is known (after the first pass). This method, however, restricts the number of forward references allowable in any expression to one.

Other special purpose procedures and functions allow symbol table insertion, location counter definition and output of listing, code and error messages.

XMETA Structure.

The XMETA system is split into two Pascal programs each about 1200 lines long. One of the programs, XCOMP, compiles assembler descriptions into an intermediate, stack-based code. This is done because it would be inefficient for the main assembler to have to compile the assembler description file each time a source program is assembled. By precompiling assembler descriptions this overhead is avoided. As well as producing an intermediate code file, XCOMP also produces a list of all the level 0 (i.e., highest level) procedure names.

The second program, XASM, performs the assembly process. There are three main sections to this program:

1. A syntax analyser which analyses assembly source statements (see section 4.6.2.).
2. A semantics analyser which analyses the 'basic' semantics and encodes them into attributes (see section 4.6.3.).

3. An interpreter of the intermediate code corresponding to the assembler definition (this is the code generator - see section 4.6.4.).

The syntax and semantic analysers generate data structures from which the predefined procedures described above can be evaluated. The intermediate code which is interpreted can only access the results of the syntax and semantic

```
   processor
identification     number of instructions
        ↘                ↘
        6800             197

        1       NOP     01      0       2       No Operation
        2       TAP     06      0       2       CCR = A
        3       TPA     07      0       2       A = CCR
        4       INX     08      0       4       X = X + 1
        5       DEX     09      0       4       X = X - 1
        6       CLV     0A      0       2       V = 0
        7       SEV     0B      0       2       V = 1
        ↖       ↖       ↖       ↖       ↖       ↖
     meaningless mnemonic opera- operand execution comment
       sequence            tion   type     time
        number             code
```

Figure 14. *Sample part of CASS-8 Assembler Description File.*

analysis via these predefined procedures. Other semantic structures (such as allocation counters, flags, etc.) are kept within the interpreter's global data area. Figure 22 shows the flow of control within the complete XMETA system.

The operation of the XASM program consists of a loop between the analysers and the interpreter. For every statement in the source program, the syntax and semantics analysers are employed to fill data structures with the decoded statement. If no errors occur during the analysis phase, control is passed to the interpreter which calls up the intermediate code procedure with the same name as the command in the assembly language source statement. This is interpreted and the remaining assembly actions are performed by the interpreter (code generation, listing control, etc.). At completion of the intermediate code procedure control is passed back to the analysers, which fetch the next source statement. This process is summarised in Figure 23.

Implementation Experiences.

The authors of the XMETA system have demonstrated its effectiveness and portability by implementing a number of assembler definitions and by transferring the system between different host computers.

Implemented processors : TMS1000, M6800, I8048, TMS9900, MC68000 and Z8000.

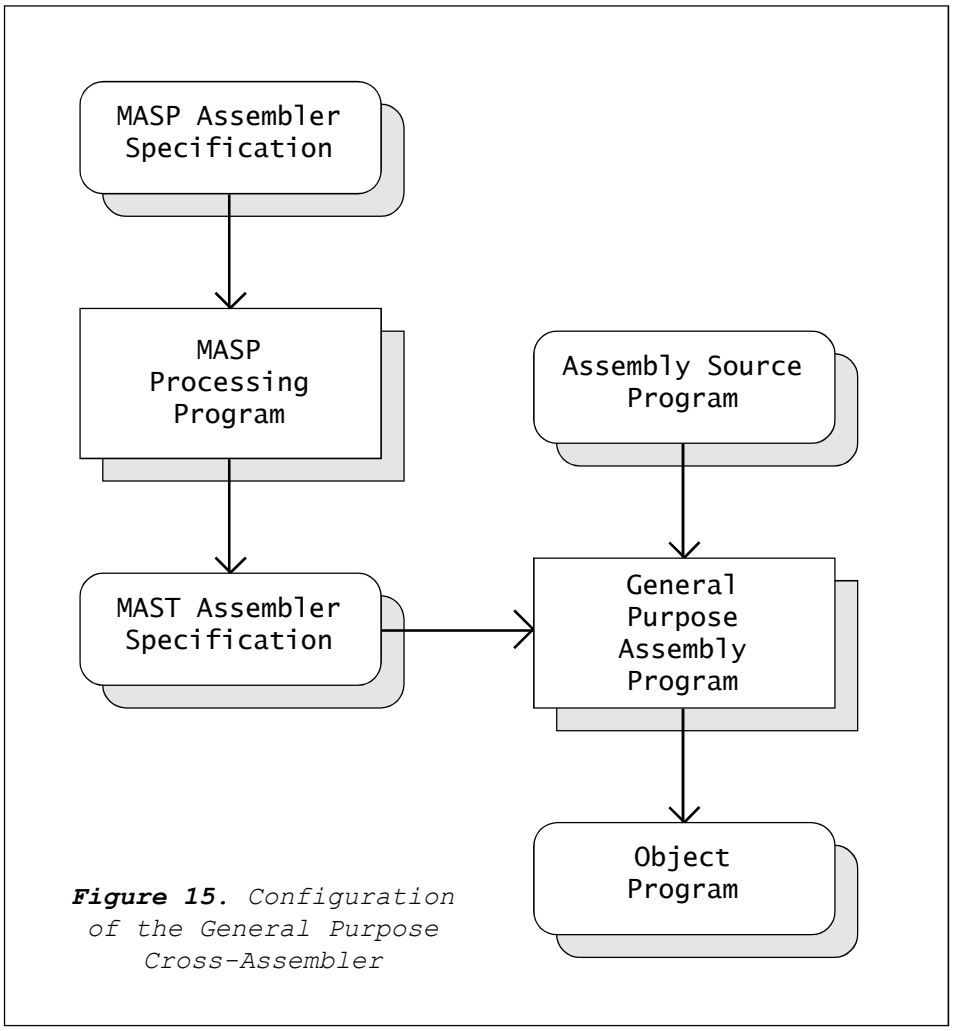


Figure 15. *Configuration of the General Purpose Cross-Assembler*

Supported host computers: PDP11/03, /23, /40, VAX11/780, PDT11/150, MINIMINC and microprocessor development systems.

The ease with which assembler definitions can be written is proportional to the experience gained using the system. An experienced XMETA programmer can code an assembler definition for a new processor in a few hours, most of which is spent reading documentation. The amount of debugging and testing time depends greatly on the

Sort	Microprocessor Name
4 bit	4004, 4040, uCOM4, uCOM41, PPS-4
8 bit	8008, 8080, M6800, MB8861, F-8, PPS-8, SCAMP, Z-80, COSMAC (CDP 1801/02)
12 bit	TLCS-12, TLCS-12A, IM6100
16 bit	PFL-16A, uCOM-16, LSI-11, TMS9900, CP-1600, PACE

Figure 16. *Microprocessors to which the General Purpose Cross-Assembler is applicable*

orthogonality of the target processor and the complexity of the object code required. Detection of some errors also increases development time. By partitioning machine instructions into appropriate format classes (see section 4.5.2.) the debugging and testing time was reduced to an average of two days, which is comparable to the time required to debug and test a Pascal program of the same length.

Sort	Microprocessor Name	Number of Source Statements
4 bit	4004	80
	uCOM-4	59
8 bit	8080	110
	M6800	266
	F-8	143
12 bit	TLCS-12A	169
16 bit	PFL-16A	257
	TMS9900	204

Figure 17. *Number of MASP Source Statements required to describe an assembler.*

The length of assembler definition programs compares favourably to the length of dedicated assemblers for the same target processor. A ratio of 3/5 was obtained when comparing with a dedicated cross-assembler written in Pascal for the TMS1000 processor, and a ratio of 1/4 was obtained when comparing with a dedicated cross-assembler written in XPL for the TMS9900 processor.

The speed of execution also compared favourably with that of a dedicated, two-pass cross-assembler thanks to pre-compiled assembler definitions and a one-pass assembly stage (with a quick fix-up scan for forward reference instructions).

4.7.6. MAUFI - A Meta-Assembler with a User Friendly Interface.

MAUFI[15] is an adaptive, non-Ferguson type meta-assembler which is table-driven. Unlike other table-driven meta-assemblers, however, the method used to enter assembler descriptions is user friendly. This reduces both the time and skill necessary to enter an assembler description. Figure 24 shows the structure of MAUFI.

PREPROCESSOR.

The preprocessor presents the user with a number of forms which together are used to form an assembler description. The selection of forms presented to the user varies depending on the users response to previous forms. The preprocessor builds up a database about the processor being described to it. This database is then used to configure the assembler stage so as to perform the correct translation process for the processor described.

The preprocessor groups the assembler description into a number of sections:

1. Code arrangement.
2. General syntax.
3. Constant syntax.
4. Registers/flags.
5. Addressing modes.
6. Instructions.
7. Pseudo operations (assembler directives).

Figure 25 shows the form for the Code Arrangement section. Some forms, such as this one, simply consist of fields which must be entered with the correct information. The Addressing Modes form, on the other hand, requires more

detailed information in the form of simple equations.

Instructions are entered into the preprocessor one at a time and no preliminary grouping is necessary. For each instruction, the following information is required:

1. Mnemonic.
2. Machine code instruction bit mask.

```
          0       Contains the value of the maximum user's allotted
                  memory address (e.g., 65526).

         1-5      Reserved for future assembler directives.

         6-25     Various microprocessors register equivalent values
                  and equivalent codes for memory addresses (e.g.,
                  A 0000007 for Intel 8080).

          26-     Each element contains:
                  Characters 1-4:         mnemonic code
                  Characters 5-7:         opcode value
                  Character 8:            process code for first
                                          operand variable
                  Character 9:            process code for second
                                          operand variable

          26-     Characters 10-11:       instruction group code
                                          number*

     (last-1)     ZZZ 0000000:            for illegal instruction
                                          processing

       last       **:                     to indicate end of the
                                          data set file
```

```
*e.g., MOV 0643003

where,
  Characters 1-4        MOV is mnemonic name for the instruction MOVE
  Characters 5-7        064 is the opcode for MOV (base opcode)
  Character 8           3   is first process code for MOV, i.e., it will
                            multiply the first operand variable by 2^3
                            (first process code, i.e., 3) and add to the
                            opcode value.
  Character 9           0   is the second process code for MOV, i.e., it
                            will multiply the second operand variable
                            by 2^0 (second process code, i.e., 0) and
                            add to the opcode value.
  Characters 10-11      03  is the group code, i.e., this is 1-byte/2-
                            operand-type instruction.


Following is an example of the actual assembling process for MOV:
LOOP: MOV       A,D     ; for Intel 8080 microprocessor
                        ; register A = 7 and register D = 2

Assembled opcode for the MOV  A,D instruction equal to assembled opcode =
(064 + 7*2^3 + 2*2^0)10 = (122)10 = (7A)16
```

Figure 18. Format of the assembler definition file.

3. Operation of instruction: Byte/Word/Long.
4. Number of operands.
5. Operand types.
6. Instruction length.
7. Instruction format.

ASSEMBLER.

The assembler is organised in the traditional two-pass configuration, pass one being concerned with defining all symbols and pass two doing the code generation.

An implementation of MAUFI has been written by its authors in the high level language C on a microcomputer based Unix system. A number of assembler specifications have been written, such as for the Zilog Z8000 and Motorola MC68000.

4.7.7. A Generative Approach to Universal Cross-Assembler Design.

The latest report to be published on meta-assemblers (to my knowledge) is written by Peter Chiu and Sammy Fu of the Hong Kong Polytechnic[32]. They present a radically different approach to the meta-assembler problem. The approach incorporates high level language development tools and considers assemblers as high level language compilers. The result is an adaptive, compile-time dedicated meta-assembler, the structure of which is shown in figure 26.

By defining syntax rules at the source program stage (i.e., compile-time dedication) a much more flexible syntax can be obtained, enabling the production of assemblers which can accept the same source programs as a dedicated assembler. Two development tools, YACC and LEX, are used to simplify the generation of lexical and syntax analysers. Included with the 'modules' which contain the lexical and syntax analysers are other modules which can be placed into two broad categories. These are 'assembler-dependent' and 'assembler-independent' modules. To make up a complete assembler the following modules are required:

1. Main overall control module - independent.
2. Lexical analyser - dependent (on the assembly language).
3. Syntax analyser - dependent (on the assembly language).
4. Symbol table manipulation module - independent.

5. Error handling module - independent.
6. Instruction set module - dependent.
7. Directive handling functions module - dependent.
8. Coding module - independent.

The lexical and syntax analysers are dependent on the assembly language structure of the source program. It is possible with this system to write two assemblers for the same processor with different assembly languages by changing just the analyser modules and equally possible to use the same assembly language for two different processors (if this is physically possible).

Two primitive assemblers have been implemented by the authors of the report for the Intel 8086 and the Motorola MC68000. The results of extensive testing on the assemblers compared with dedicated assemblers and other universal cross-assemblers are shown in figure 27. It can be seen that the assemblers produced are 4 to 5 times slower than comparable dedicated assemblers and 2 to 3 times slower than comparable universal cross-assemblers.

Implementation time for a new assembler is quoted as about 20 to 30 days which is considerably longer than other implementations discussed earlier (although it is still substantially less than the time required to write a dedicated assembler).

5. A SAMPLE IMPLEMENTATION.

This section of the report details the design and implementation of a sample meta-assembler system.

5.1. Problem Definition.

The problem which, as defined at the start of the project, was

'to develop a meta-assembler with the following properties:

* Portable.
* Easy to use.
* Non-restrictive.
* General.'

For a more detailed description of these properties refer to section 4.2.

5.2. System Analysis.

The purpose of the system analysis phase is to describe what is to be achieved and put forward suggestions of how this can be achieved. By doing background research into the problem, which was the concern of section 4,

1. The problem is well defined.
2. A number of alternatives for solving the problem have been suggested (and in most cases, implemented).

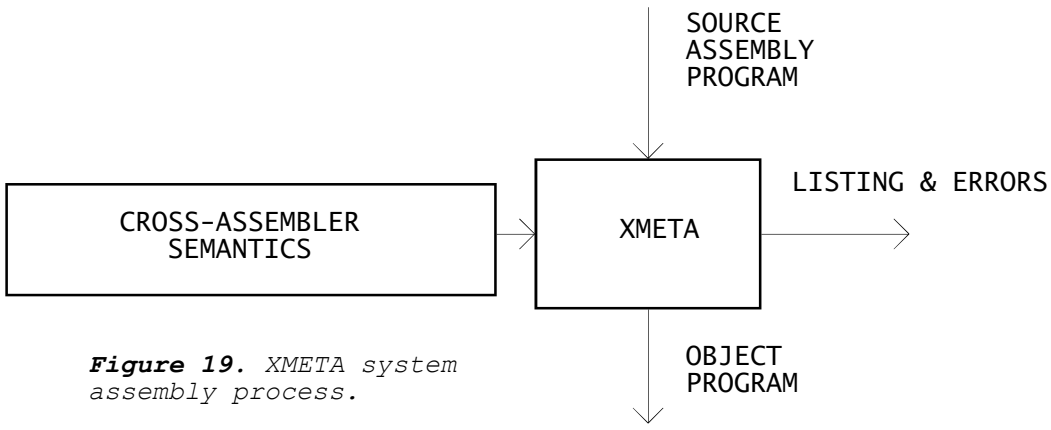


Figure 19. *XMETA system assembly process.*

There would be little point in repeating the information in section 4. The main task for the system analysis phase of the software cycle in this case, therefore, is to select an appropriate method of solving the problem, justifying that decision.

5.2.1. Approach to Solution.

One of the major decisions which must be taken when selecting a method of solving the meta-assembler problem is what form the assembler description/specification should take. There have been two possibilities suggested:

1. Tables. The assembler description takes the form of one or more tables.
2. Languages. The assembler description takes the form of a program in a language which may be either macro-

based or high level language-based.

When looking at these two choices conceptually they can be summarised independently of the physical form used to represent them.

1. Data-driven.
2. Algorithm-driven.

Both of these possibilities have their advantages and disadvantages. The data-driven approach is simpler and quicker to implement but is not very flexible while the algorithmic approach is very flexible but more complicated to implement. There is, however, a third choice which combines the benefits of both approaches while largely eliminating their weaknesses. This approach is

3. Data and Algorithm-Driven.

It might at first appear that XMETA (see section 4.8.5) falls into this category, but it is strictly algorithm-driven because variables of any kind are not allowed in the assembler description program (as with most language-driven implementations, only parameters are allowed). The most recent implementation of a meta-assembler[32] does take this approach and the benefits are apparent in the flexibility which is provided (see section 4.8.7.). The way in which this particular implementation approaches the problem, however, means that only certain sections of the solution can be truly classed as data and algorithm-driven.

It is therefore proposed that the sample implementation should take a data and algorithm-driven approach.

The simplest solution to the problem using this approach was suggested by the authors of the XMETA system in their report 'A High-Level Language Based System for Cross-Assembler Definition'[23].

Every dedicated assembler which is written is data and algorithm-driven. This has always traditionally been the best method of writing an assembler for one processor. The problem when considering how to reconfigure a dedicated assembler so that it can assemble another source language is that the assembler-specific part is spread throughout the whole of the source program. This is usually the case because it allows for the fastest implementation. The solution to this problem, therefore, is to separate out the assembler-specific and general parts of a dedicated assembler and make them as independent as possible. It will be seen that there is a trade-off between making some sections assembler dependent (which will mean new assembler descriptions are easier to implement) and the generality of the resulting meta-assembler.

```
[label:] command [[prefix] argument [(index [,index])]
[postfix][,...]] [;comment]

Ident(0) Narg    Pref1(i)  Arg(i)     Xf1(i)  Xf2(i)    Post(i)
```

Figure 20. *XMETA Predefined Functions and Procedures.*

Once the outline design had been chosen, the choice of which language to program the implementation in had to be made.

5.2.2. Selection of Programming Language.

A number of factors affected the decision of which programming language to use when writing the sample implementation. These were (in order of priority):

1. A language which was either familiar or easy to learn.
2. Available on a large range of computers (microcomputer development systems upwards) and be portable in nature.
3. Suitable for programming a large software project. Provision for decomposing a problem into separate, manageable pieces.

```
ALFA:   MOV     @BETA+1(R3),R2+

Ident(0) = 1; Symbol(0) = 'ALFA';
Narg = 2; Nforw(1) = . . . (depends on the context);
Pref1(1) = ord('@'); Ident(1) = 1;
Arg(1) = 1; Type(1) = forward;
Symbol(1) = 'BETA'; Xf1(1) = 3 (supposing Ri = i);
Pref2(1) = Pref3(1) = Xf2(1) = Post(1) = nul;
Ident(2) = 1; Arg(2) = 2; Type(2) = Absolute;
Symbol(2) = 'R2'; Post(2) = ord('+');
Pref1(2) = Pref2(2) = Pref3(2) = Xf1(2) = Xf2(2) = Nforw(2) = nul;
Symbol(i) = '        ' for i > Narg;
any other function with argument > Narg = nul.
```

Figure 21. *Examples of predefined function evaluations (assuming BETA has not yet been defined).*

4. Support in some way for the principles of Object Oriented Software Construction.
5. Facilities for low-level, machine-oriented manipulations.
6. Fast enough to produce a usable product.
7. Good development environment with extensive debugging facilities.
8. Be available on the computer which is to be used for the implementation.

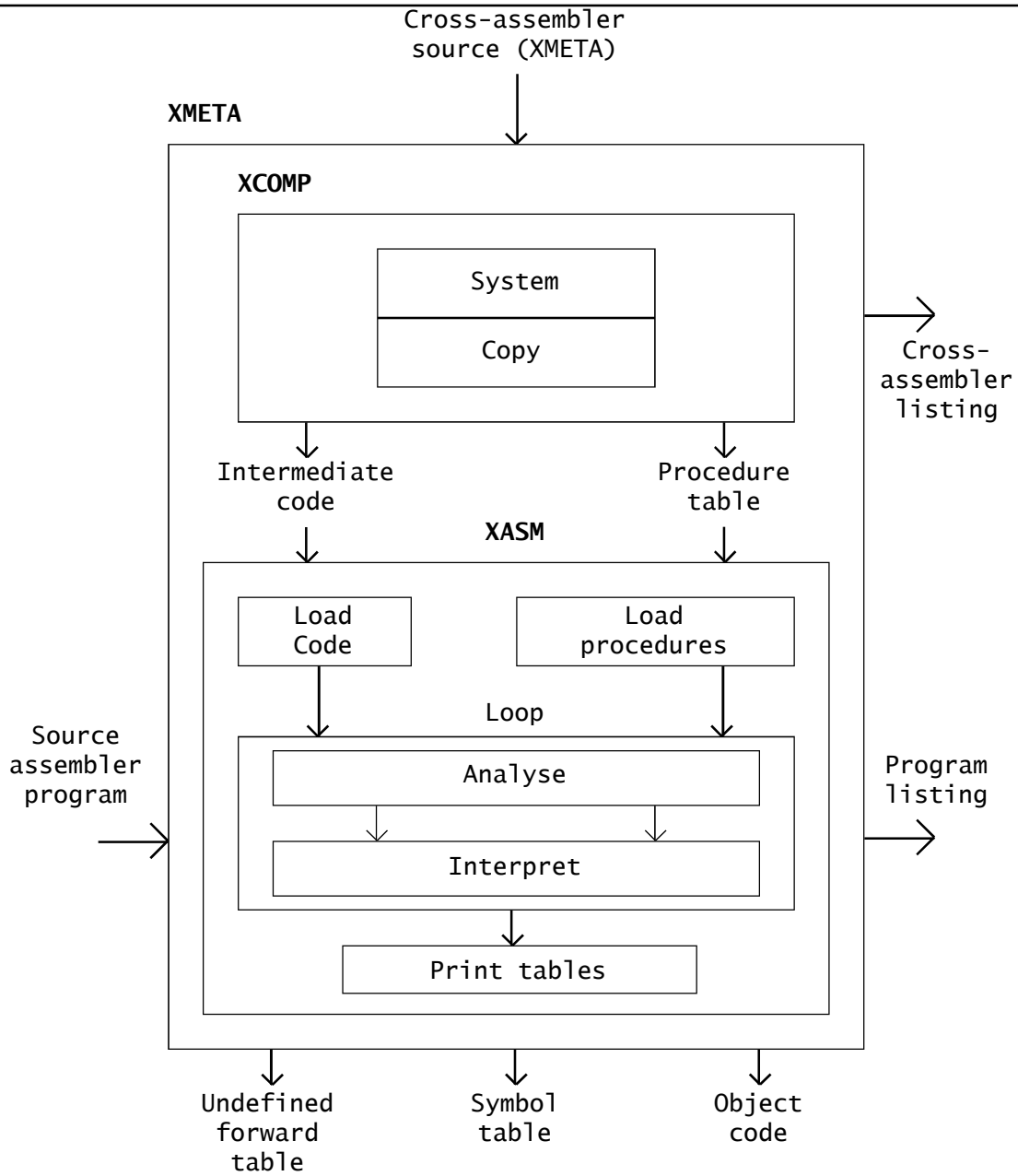


Figure 22. *Flow of control in the XMETA system.*

The languages which were considered against this criteria were:

1. C.
2. C++.
3. Smalltalk V.
4. Modula-2.
5. Pascal.

All of these languages are available for the IBM XT computer which was to be used. The language which was chosen was Modula-2 because it is the best at meeting all the requirements specified. C was ruled out because it does not have very good facilities for large program structuring, has no Object Oriented Programming (OOP) facilities and was not very familiar. C++ (an OOP extension of C) was ruled out because it is even more difficult to learn than C. Smalltalk V, although having an excellent development environment and providing good support for OOP, does not produce code which is practical (in terms of speed and size) for this sort of application. Pascal was ruled out because it provides no facilities for OOP or large program structuring. Modula-2 gave the following answers to the selection criteria:

1. Modula-2 was a familiar language as it was the programming language chosen for the 'Programming and Programming Design' course in year 2 and for the 'Computer Systems' course in year 3.
2. Modula-2 development systems are available for virtually all well known computers, and it is small enough to be used on microcomputer development systems.

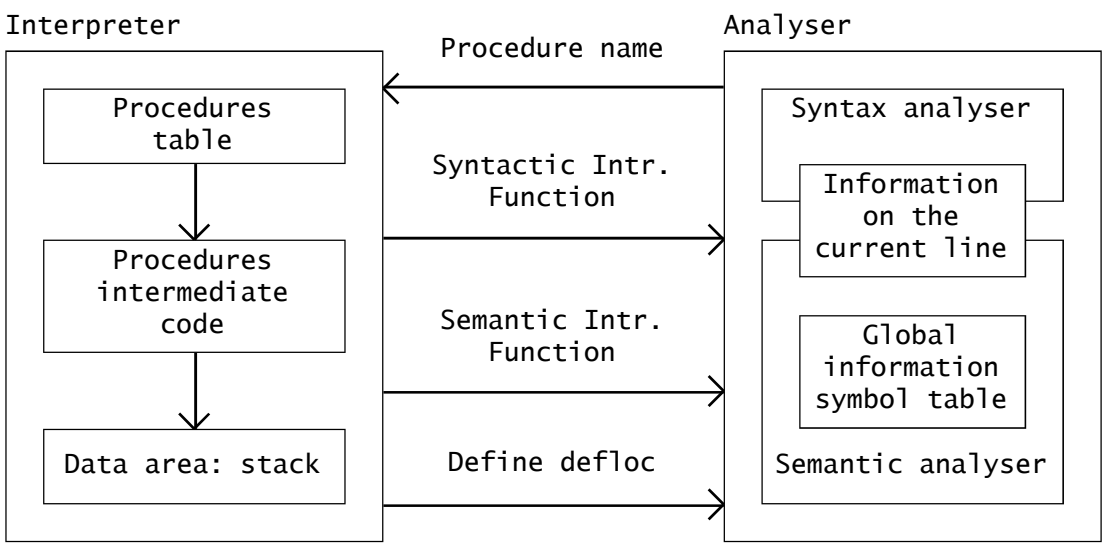


Figure 23. *Information flow within XASM*

3. Very good facilities are provided for decomposing large programs in manageable chunks via the 'MODULE' structure.
4. Although support for OOP in Modula-2 is limited, an OOP approach to the design process maps well onto implementations.
5. Good low-level facilities are provided via the bitset type and associated set operations.
6. Recent benchmarks place Modula-2 compilers at the top in terms of generated code efficiency.
7. The implementation of Modula-2 chosen, JPI Topspeed Modula-2, has an excellent integrated environment with extensive debugging facilities.
8. JPI Topspeed Modula-2 is available for the complete range of IBM-compatible computers.

5.3. System Design.

This phase of the software life cycle is concerned with the overall structure of the product and how, generally, it will solve the problem definition.

5.3.1. Decomposition into Modules.

A useful decomposition of a traditional two-pass assembler into a number of modules is given by Gorsline in his book 'Assembly and Assemblers'[1]. The module titles are:

1. Input/output module.
2. Decoder from free-format module (lexical scanner).
3. Table entry/search module.
4. Location counter adjustment module.
5. Error handler module.
6. Assembler directive processor module.
7. Machine-code generator module.
8. Expression evaluation module.
9. Listing module.
10. Coordination (executive) module.

The next task is to decide which of these modules can be assembler-independent. Trade-offs between generality and ease-of-use/ease-of-implementation will have to be made.

It can be seen that the following modules can be assembler-independent without any trade-offs being made:

Input/output module.

Table entry/search module.

Location counter adjustment module.

Listing module.

Error handler module.

Coordination module.

For the other modules, selection of whether the modules should be assembler dependent or independent will affect the performance of the meta-assembler.

Decoder from free-format module (lexical scanner).

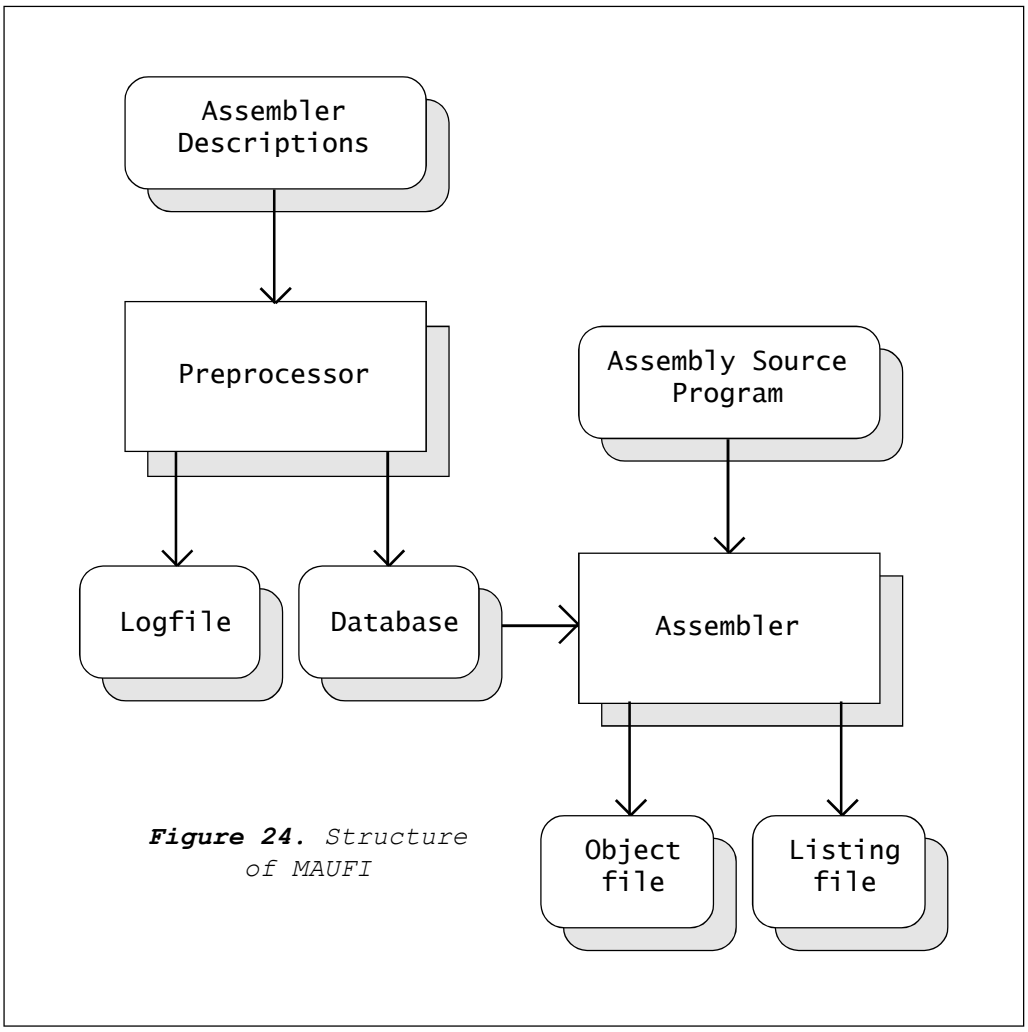


Figure 24. *Structure of MAUFI*

If this module is made assembler-independent the syntax of the assembly language statements for any assembly language must be fixed. This would mean that the meta-assembler could not mimic any dedicated assemblers and existing software would need converting from the dedicated assembler's syntax to the meta-assembler's syntax. The alternative is to supply the syntax of the assembly language in the assembler description. This has been done[32] but requires larger assembler descriptions and a more complicated meta-assembler.

It was decided that this module should be assembler-independent because for the situations where a meta-assembler is most useful the fact that all assembly languages must have the same source syntax is actually an advantage. The need to mimic dedicated assemblers is not critical in these situations.

Assembler directive processor module.

If this module is made independent of the assembly language then the set of directives which are implemented are fixed, whatever assembly language is being assembled. If this module is supplied as part of the assembler description then assembler directives can mimic those of dedicated assemblers.

```
Section: Code Arrangement          name

Alignment (odd/even)             : xxx
Word length                      : xx
Bit arrangement (0/15 LSB)       : xx
Byte arrangement (n,n+1/n+1,n)
                 (  1  /  2  )   : x
Data arrangement (High/Low)
                 (byte first)    : x

Any modifications (y/n) ?
MESSAGE:
```

Figure 25. *Code Arrangement Form.*

For the same reasons given for the lexical scanner, it was decided that this module should be assembler-independent. In that way, assembler directives are constant throughout the range of assemblers implemented.

Machine-code generator module.

This module really performs two tasks: Generation of the machine-code corresponding to an assembly language statement and writing the generated machine-code into an object code file.

The first task is definitely assembler dependent, but there is a choice with the second. If this task is made assembler dependent any format of object code can be produced. If this task is made assembler-independent, the format of the object code produced is fixed.

It was decided that a compromise between the two alternatives would provide the best solution, by making the object code generator assembler-independent but providing it with a number of alternative object code formats. This means that if there isn't an object code format provided which is suitable, either one can be added to the assembler-independent part of the meta-assembler (not necessarily the best solution) or a small conversion program could be written which converts a generic format of object code into the one required.

This module is therefore split into two separate modules:

1. Assembler definition module (ADM). This contains the target processors parameters and the machine code generator.

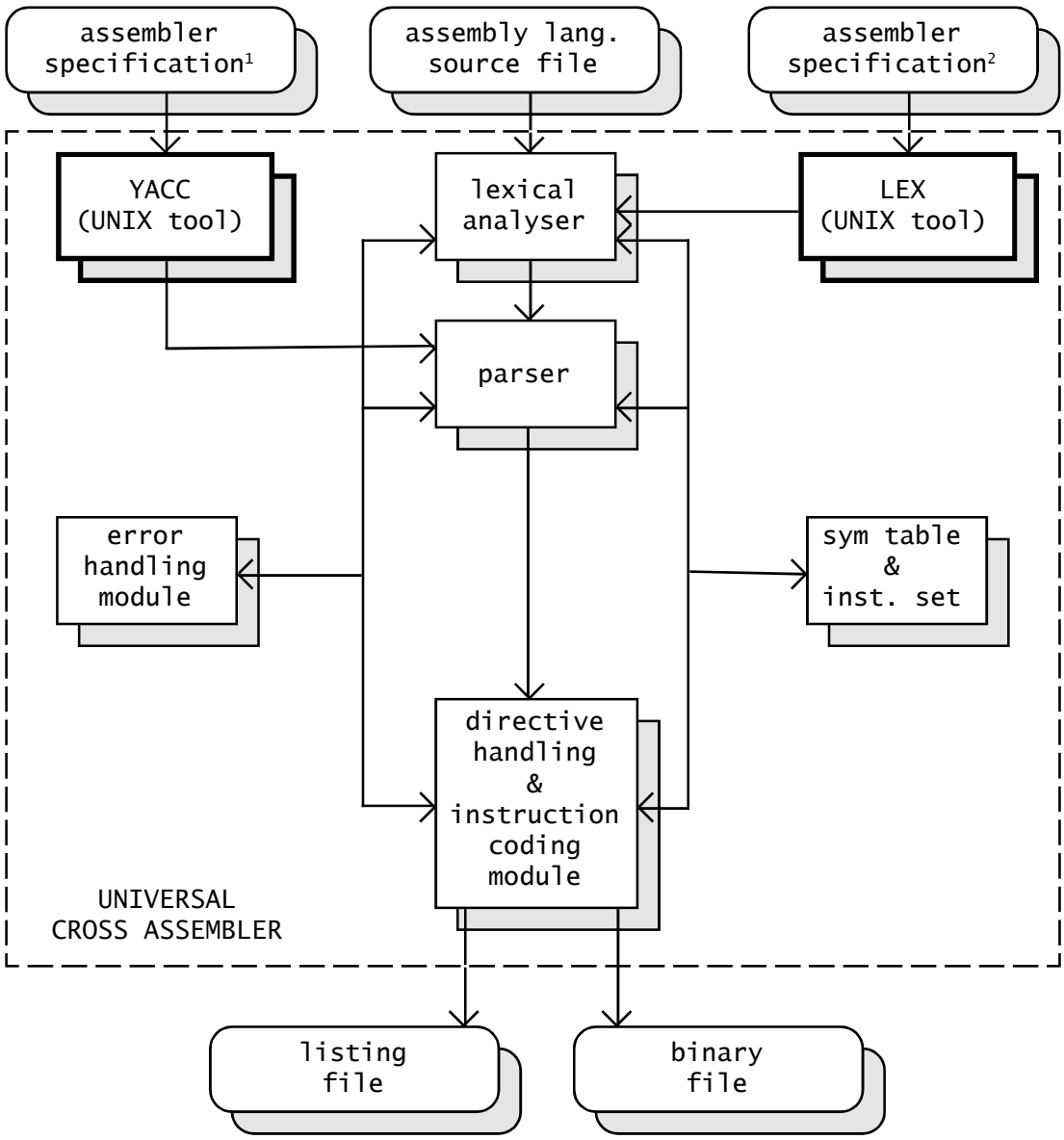
2. Object code generator module. This contains conversion routines which convert the standard machine code generated by the ADM into an object code format.

To summarise then, the set of main modules which make up the sample implementation are:

1. ADM - Assembler definition module.
2. Exceptions - Error handler module.
3. Expression - Expression evaluator module.
4. Interface - Input/output module.
5. Lex - Free format decoder module.
6. Listing - Listing file generator module.
7. ObjectGenerator - Object code file generator module.
8. PseudoOps - Assembler directive processor module.
9. Table - Table entry/search module.
10. M2Assem - Coordination (executive) module.

5.3.2. A General Model for Assembly Languages.

As the decision had been taken that the lexical scanner module should be assembler-independent, a general model for assembly languages had to be developed as a specification of the syntax of any source language which could



1 - syntax rules
2 - lexical rules

Figure 26. *Structure of UCA.*

be input. Rather than develop a model for all assembly languages, five were chosen to start with, so that the model was not initially over-complicated. The five processors were MC68000, 6800, 6502, Z80 and 80x86. The manufacturer's assembly languages were selected in each case.

It was found by examination that all of the assembly languages had the same general format for an assembly language statement:

```
[label]     opcode     operand(s)     [comment]
```

Similarly, assembler directives also conformed to a general format:

assembler (for which uP)	no of source lines	time (sec)	speed (lines/sec)
Dedicated assemblers (from CROSS-8 package)			
6805	271	25	10.84
8051	308	25	12.32
6804	100	13	7.69
6800	350	30	11.67
8085	880	70	12.57
Z80	870	67	12.98
Microsoft MASM	130	7	18.57
2500 A.D. 68000 Assembler	200	15	13.33
Universal Cross Assembler (from CROSS-8 package)			
6502	120	32	3.75
6805	100	25	4.00
8085	48	13	3.69
8048	80	37	2.16
Z80	250	75	3.33
6801	216	40	5.4
8051	120	40	3.0
Chiu & Fu Implemented Assemblers			
8086/88 COMASM	130	50	2.60
68000 X68000	200	73	2.74

Figure 27. *Assembly speed statistics.*

```
[label]     directive  argument(s)    [comment]
```

The differences in the syntax of the languages came when examining the individual fields of a statement.

Label Field.

The label field of all the assembly languages has to start in the first column of a source program and consists of an alphabetic character with alphanumeric characters following it. The number of significant characters varies between assembler implementations and is processor independent.

Opcode Field.

All opcodes consist of an alphabetic mnemonic which is always less than 5 characters in length. The Motorola MC68000 assembly language opcode is followed by a modifier which indicates the size of the operation:

.B = 1 byte.
.W = 2 bytes/1 word.
.L = 4 bytes/1 longword.

Amongst the processors chosen, any operation without a modifier defaults to a word size operation. This format is unique to the MC68000.

Operand(s) Field.

The operand(s) field of all the assembly languages all contain both an indication of the addressing mode being used and the values to be operated on. These values can take a number of forms:

Expressions - any set of constants can have either arithmetic or logical operations performed on them within the operand of an instruction.

Constants - represent the value of either a fixed value/attribute or a location in the source program. Examples are:

- Registers.
- Program counters.
- Condition codes.
- Stack base pointers.
- Operand sizes.
- Specific memory/IO locations.

Addressing Specifiers - expressions and constants are contained within addressing specifiers. These are formed using special ASCII characters.

Examples of Addressing Modes for Operands.

MC68000 - `#constant  Dn  An  (An)  (An)+  -(An)  constant(An)  (constant,An)`
`constant(An,index,constant.specifier)`
`where n is a number from 0 to 7.`

6502 - `#constant  constant  (constant,X)  (constant),Y`

Z80 - `(register)  constant  (register+constant)  (constant)`

6800 - `#constant  constant  X  (constant)`

80x86 - `register  constant  [constant]  [register]  specifier[constant]`

Addressing mode modifiers are used within two of the assembly languages (68000 and 80x86) to specify or clarify the size of a particular operand. For each of these languages, defaults are assumed if these are omitted.

Addressing Modes.

The syntax of addressing modes can be seen to vary widely between assembly languages. The provision for all these formats of addressing modes would certainly lead to a complicated lexical scanner. The alternative to this is to formulate a set of standardised addressing modes with which all the assembly languages can be mapped onto. This produces an easier implementation and more elegant solution but would mean that some conversion of

manufacturer's assembly languages to the standardised assembly language would be needed.

By initially basing a standardised set of addressing modes on the MC68000 syntax, it was possible to draw up syntax diagrams of a general assembly language model (for these five processors). A summary of the syntax diagrams is shown in Backus-Naur form in figure 28.

5.3.3. The IEEE Assembly Language Standard.

The work carried out in section 5.3.3. was done prior to receiving a report by W. Fisher entitled 'Microprocessor Assembly Language Draft Standard'[25]. In this report, a standardised model of assembly languages is created by following a number of guidelines. Using this method, it is possible to implement an assembly language for virtually any processor using a completely standardised syntax. This was obviously more desirable than the limited syntax which was developed in the previous section. It was therefore decided that the sample implementation to be developed should have a lexical scanner which would accept any assembly language program conforming to this IEEE standard. This means that the implementation will not be able to accept source programs written using a manufacturer's assembly language syntax without some modification (quite extensive in some cases). This limitation is not severe, however, when the use of meta-assemblers is put into context. Section 4.7.2. pointed out that there were two main areas for which meta-assemblers are generally developed:

1. Educational establishments.
2. Small system developers.

For both of these areas, apart from the initial increase in activity due to converting existing software to the new syntax (which might be accomplished by a translation program automatically), the use of a generalised syntax is a productivity aid in that all programs developed have the same syntactic structure and the same set of directives (which are part of the defined standard). The operation of the meta-assembler will always be the same, so familiarity with it will increase instead of the decrease normally experienced when many dedicated assemblers are used within a short space of time.

<u>Features of the Standard</u>.

The IEEE Standard covers the following areas of assembly languages:

* Instruction Names.

* Mnemonics.
* Addressing Modes.
* Operand Sequences.
* Expression Evaluation.
* Constants.
* Labels.
* Comments.
* Assembler Directives.
* Standard Instruction Sets.

Instruction Names.

The standard defines a large set of standard instruction names which are to be used when the comparable instruction exists in an assembly language. If an instruction does not have a comparable standard name, one should be invented: instruction names shall begin with an action verb.

Instruction Mnemonics.

A set of rules is given for the mnemonics of instructions which are not contained within the standard. These are:

1. The first character of a mnemonic *shall* be the first letter of the action verb.
2. Addressing modes *shall not* be embedded in the mnemonic.
3. Operand designations *shall not* be embedded in the mnemonic.
4. Conditions *shall* be embedded in the mnemonic.
5. Operand type may be indicated, where appropriate, by the last character of a mnemonic, as shown below (the default operand type is word):

   B: Byte.
   H: Halfword.
   L: Long (Double Word).
   D: Decimal.
   F: Floating Point.
   1: Bit.
   4: Nibble or Digit.

M: Multiple.

Also given is a complete set of modifier extensions used to indicate conditional instructions.

Addressing Modes.

A set of addressing modes is defined in the standard. Addressing modes are specified by adding special characters to the operands. The following prefix and postfix characters are used:

Mode	Symbol	Example
Absolute	prefix /	/addr
Base page	prefix !	!addr
Indirect	prefix @	@addr
Relative	prefix $	$addr
Immediate	prefix #	#value
Index	enclosing parenthesis ()	addr(index)
Register	prefix .	.addr
Auto-pre-increment	prefix +	+addr
Auto-post-increment	postfix +	addr+
Auto-pre-decrement	prefix -	-addr
Auto-post-decrement	postfix -	addr-
Indirect-pre-indexed	prefix () @	addr(index)@
Indirect-post-indexed	prefix @, postfix ()	@addr(index)

Operand Sequences.

The proper sequence of operands in any particular instruction is given in the standard.

Move operands. The source of data is defined to be the primary operand. The operand sequence shall be: source, destination.

Load and Store operands. The register to be loaded or stored is defined as the primary operand. The operand sequence shall be: register, memory.

Arithmetic operands. In two operand instructions, the operand that is both the source and destination is the primary operand, and the second source is the secondary operand. In three operand instructions, the result is defined to be the primary; therefore the destination shall be the first operand, followed by the "first" source, then the "second" source.

Expression Evaluation.

When expression evaluation capabilities are included in an assembler, the expression operators shall be designated by the following special infix symbols:

Symbol	*Operation*
+	Add
-	Subtract
*	Multiply
/	Divide (Signed)
//	Divide (Unsigned)
.AND.	Logical AND
.OR.	Logical OR
.XOR.	Logical Exclusive OR
.NOT.	Logical NOT
.SHL.	Left Shift
.SHR.	Right Shift
.MOD.	Modulo
**	Exponentiate
<:>	Bit alignment

Constants.

All numeric constants shall be represented by an alphabetic character specifying the base followed by an apostrophe. The following characters are reserved for the specified base:

Character	*Base*	*Example*
B	Binary	B'1001111
Q	Octal	Q'117
D	Decimal	D'79
H	Hexadecimal	H'4F

A default is assumed if no specifier is given. This can be changed using the BASE assembler directive.

Character strings shall be enclosed within quotation marks:

"THIS IS A CHARACTER STRING"

The special symbol * shall be used to refer to the location counter.

Labels.

All labels shall begin with an alphabetic character in the first column on a line. At least one blank must separate the label from the rest of the line. Labels may optionally be terminated with a colon.

Comments.

Comments may appear after a statement on the same line, or on a separate line. Each comment shall start with a semi-colon. For example,

```
; This is a comment
    BZ    NEXT  ; Comment on same line
```

Assembler Directives.

The naming of assembler directives shall follow the rules given for instructions. If the following functions are implemented, the specified mnemonic shall be used.

Mnemonic	*Function*
ORG	Set location counter to expression
EQU	Equate label to expression
END	Mark end of source program
PAGE	Advance page in listing file
TITLE	Set title in listing file
DATA	Fill memory with specified data
RES	Reserve memory of specified size
BASE	Change default expression base

Standard Instruction Sets.

Standard instruction mnemonic sets are provided for the following processors:

Intel 8086
Motorola 6800
Zilog Z80, Intel 8080 and 8085
DEC LSI-11
Motorola MC68000
Zilog Z8000

5.4. Detailed Design.

The approach taken towards the detailed design of modules was to design them with the principles of Object Oriented Programming in mind. This particular design philosophy was chosen because it is the way in which software engineering as a whole is turning because the principles behind it lead to the generation of 'better' software. By 'better' it is meant that software is more robust and reliable. The method used to achieve this is by breaking a system down into a number of objects which are independent of their environment. The objects encapsulate data structures and procedures which operate on them.

Frank and Hewitt's book entitled 'Software Engineering in Modula-2' quotes a number of claims which are

typically made for Object Oriented Systems:

* Increased comprehensibility.
* Guaranteed data integrity.
* Enable implementations to be easily changed.
* Provide data independence.
* Encourage independent development (of objects and collections of objects).
* Reusability and extendability.

It can be seen that this design philosophy is very suitable for the development of meta-assemblers which, in themselves, have to be very adaptable. It is the very fact that traditional dedicated assemblers are written by decomposition into functional modules rather than objects that leads to them being so unadaptable to other assembly languages.

Support for OOP in Modula-2 is limited. However, by combining object oriented design (which can be done independently of the programming language to be used) and Modula-2 features which support OOP, a robust and reliable implementation can still be produced.

5.4.1. Modula-2 Support for Object Oriented Programming.

This section describes the limited support that Modula-2 provides for OOP and defines some of the OOP features supported. Material was taken from 'Software Engineering in Modula-2'[12] and 'Data Structures with Modula-2'[11].

Object.

In Modula-2, each object can be implemented as a MODULE containing a data structure and a number of operations (procedures and functions) which can be performed on that structure. If the object represents a single entity then it is an *Abstract State Machine* and if the object represents a class of entities then it is an *Abstract Data Type*.

Data Structure.

There are two types of structure which can exist in an object. A *transparent data structure* is one which is completely accessible to a user of that object and can be manipulated outside the object. An *opaque data structure* is one which cannot be manipulated outside an object and is therefore more secure. Opaque data structures do have some disadvantages, however.

Operations.

Operations which can be performed on a data structure can be put into a number of classes.

1. Constructors. A constructor operation for a given object is one which creates an instance of that object from its component parts.
2. Selectors. A selector operation for a given object is one which selects a specific component part of an instance of that object.
3. Exceptions. Erroneous conditions which arise out of the use/misuse of an object can be handled in one of two main ways:

A predicate is provided which, when called, returns information about the state of an object.

An exported variable is provided which can be tested by a client program to provide information about the state of an object.

There is one other method of providing a satisfactory solution to exceptions, which is to provide exceptions themselves as an object.

Another important concept of OOP is module *reusability*. Depending on the type of software being developed, this may or may not influence the design of objects. In an assembler, most of the modules are very specific to this type of problem and are unlikely to be usable in many other programs. For these objects, designing for reusability is not important. The exceptions to this are the user interface object (input/output module) and the table object (table entry/search module). Objects of this type appear in many programs and it is worth designing them to be reusable. This involves trying to generalise their operation as much as possible and providing operations which will cover every eventuality as to how the object will be used. In practice, this leads to longer development times and objects which have more operations.

5.4.2. Detailed Design of Modules.

The following sections describe the design of the individual modules which make up the meta-assembler. The modules were designed, implemented and tested in the following order:

```
1. Strings.    |
2. Table.      |- These four modules were reused from previous
3. TableTrees. |  programming projects.
4. TableExt.   |
5. Exceptions.
6. Lex.
7. Expression.
8. Interface
9. Listing.
10. Object Generator.
11. ADM.
12. M2Assem.
13. PseudoOps.
```

Three test programs were also developed in order to test these modules. The listings for the above modules and the test modules can be found in the appendix.

5.4.3. Character String Handling Module (Strings).

This module was reused from a previously implemented version. The module implements the standard operations associated with character strings. An abstract data type, String, is defined with associated operations. The decision to implement the abstract data type transparently (rather than opaquely) was made because there was a large need to be able to define constants within other modules. This cannot be done with opaquely exported types (which is one of their drawbacks).

5.4.4. Exception Handler Module (Exceptions).

This module handles all the exceptional conditions which can occur within the operation of the meta-assembler. The module implements an object whose data structure is an abstract state machine. This approach is used because

```
Syntax:
        =  equivalent
        |  one or the other (OR)
        [] zero or one times
        {} zero or more times
        "" literal symbol
        .. OR between sequence of characters

Statement = [Label] Seperator Command [Specifier] Seperator [Operand {"," Operand}] [Seperator
Comment] EOL
Seperator = {"<TAB>"} | {"<SPACE>"}
Label = Alphabetic {Alphanumeric} [":"]
Alphabetic = "A".."Z"|"a".."z"
Alphanumeric = Alphabetic | Numeric | Special
Numeric = "0".."9"
Special = "_" (dependent on implementation)
Command = Alphabetic {Alphabetic}
Specifier = ["." Alphabetic]
Operand = {Value | "(" Value ")" "+" | "-" "(" Value ")" | "[" Value "]" | "#" Value}
Value = Alphanumeric [Alphanumeric] [Specifier] | Expression
Expression = dependent on implementation (includes logical and arithmetic ops)
```

Figure 28. *A Generalised Model of Assembly Languages.*

only one exception can be raised at any time. There are three possible levels of exception which can be raised:

Warning. Exceptions at this level cause no change in the operation of other modules. A message is produced in the listing file at the current line and at the console.

Error. Exceptions generated at this level cause output to the object code file to be terminated. An appropriate message is written to the listing file and console.

Fatal. An exception at this level causes the complete shutdown of the meta-assembler. This includes the closing of all files and the deallocation of all memory currently in use. The shutdown is effected by calling operations in every module which execute a shutdown procedure specifically for that module.

There are four possible types of exception:

Code. A code exception is generated as the result of an error occurring in the source program.

User. A user exception is generated as the result of a bad command line (for example, specifying a non-existent source file).

Environment. Exceptions of this type are generated when references to the environment of the meta-assembler are erroneous. For example, a memory allocation or IO operation fails.

Internal. Exceptions of this type are reserved for internal errors in the meta-assembler and do not occur in normal use.

As a result of the raising of these exceptions, the meta-assembler can be in one of four states:

Normal. This is the state in which the meta-assembler starts.

NoCode. The meta-assembler does not produce an object code file in this state.

NoList. In this state, no listing file is produced.

NoOutput. In this state, neither listing or object code file is produced.

Operations provided in the exceptions object are:

Raise - enables a module to raise an exception of a particular type and level and provide a suitable message.
ChangeStatus - The status of the program can be explicitly changed.
ProgramStatus - The current status of the meta-assembler is returned.
Number - The number of exceptions raised since the start of assembly is returned.

5.4.5. Lexical Scanner Module (Lex).

The purpose of the lexical scanner module is to decode a free-format assembly language into a fixed-format instruction (more details of this stage of assembly in section 4.6.1.). The format of the assembly language statement is given in the IEEE Standard which can be summarised:

```
[label[:]] command[.modifier]
[[prefix]argument[(index[,index])][postfix][,..])] [;comment]
```

where [] indicates optional fields and a space indicates whitespace.

The output of the lexical scanner is in the form of a data structure, which is defined in the assembler definition module.

Two operations are provided within this module:

ScanLine. This is the operation which performs the lexical scan. The string of characters supplied to this operation is examined one character at a time from the left hand column. Fields of the source line are identified and extracted as each character is read. Fields are identified by the comparison with termination and continuation characters. Continuation characters are those allowed in a field and termination characters mark the end of a field. The following list gives the appropriate termination and continuation characters for each field.

Field	Termination	Continuation
Label	Colon, Whitespace	Alphanumeric
Command	Whitespace, Fullstop	Alphabetic
Modifier	Whitespace	-
Prefix	-	PrefixSet
Argument	Whitespace, Open bracket	-
Index	Comma, Close bracket	-
Postfix	-	PostfixSet
Comment	End of line	-

where

PrefixSet = '/', '!', '@', '$', '#', '+' or '-'.
PostfixSet = '+' or '-'.

The operation only separates fields so, for example, it cannot check to see whether an expression is valid. Errors in the assembly language statement which can be detected are:

Label Error - The label field starts with a non-alphabetic character.
Modifier Error - The modifier character is not one of those allowed.
Command Error - The command field contains a non-alphabetic character.
Comment Error - A comment does not start with a semi-colon.
Operand Error - A complete operand is badly formed.
Index Error - An operand index is badly formed.
EOL Error - A valid statement was not read before the end of line was reached.
Too many operands error - The maximum number of complete operands allowed is exceeded. The maximum is assembler-dependent and is therefore defined in the ADM.

Too many indices error - The maximum number of indices allowed is exceeded. The maximum is assembler-dependent and is therefore defined in the ADM.

Another operation called **ScanOperand** is provided by this module. It takes as input an operand field of the decoded line and returns the IEEE addressing mode to which the operand conforms (or raises an exception).

5.4.6. Expression Evaluator Module (Expression).

The purpose of the expression evaluator module is to evaluate expressions which conform to the IEEE syntax. Expressions have the general form:

[expression operand 1] operator expression operand 2

The expression operands can be either symbolic names or constants. The evaluator therefore needs access to the table so that the value of symbolic names can be determined. The operation **Evaluate** performs this task.

The **Evaluate** operation first determines whether there is an operand in front of the operator by checking whether the first character in the input string is a '.'. If there is an operand, it is read and checked against entries in the table. If an entry is not found, the operand is checked to see whether it is a valid constant. An exception is raised if it is not. The operator is then read and checked and then the second operand is read. An operation is then called which applies the appropriate operation on the two operands. The only operation which has one operand is the .NOT. operation. If the expression is valid a LONGINT value is returned, otherwise an exception is raised (this includes operand type errors and results which are out of range). The type of the value returned depends on the types of the values of the operands and errors may be produced if an operation is attempted on two operand with incompatible types. The table below shows the operations available and the operand types which are valid.

Operation to perform	A op A	A op R	R op A	R op R
SHL,SHR	A	#	#	#
AND,OR,XOR,NOT	A	#	#	#
*	A	#	#	#
/,//	A	#	#	#
+	A	R	R	#
-	A	#	R	A

Where:

'A' indicates an ABSOLUTE operand.

'R' indicates a RELATIVE operand.

'#' indicates an invalid expression.

Other operations:

CheckRegister. This operation takes a string of characters and checks to see whether the string represents a valid register. The set of valid registers is defined in the ADM.

CurrentBase. This operation is a predicate which returns the current default base.

ChangeBase. This operation changes the current default base.

5.4.7. Input/Output Interface Module (Interface).

This module provides the only connection through which the meta-assembler can communicate with its environment. Restricting all the input and output operations to one module increases the program's portability, as only one module will need changing in the event of a transfer to a computer with a different standard IO library.

Three objects are defined in this module: File, CommandLine and Time.

The File object exports transparently the data structure FileHandle. An opaque export could not be used because it was necessary for client modules to assign constant values to the data structure. Operations which can be performed on a FileHandle are:

FileExists - Test whether a file exisits.
CreateAFile - Create a new file.
OpenAFile - Open an existing file.
CloseAFile - Close an existing file.
DeleteAFile - Delete a file.
EndOfFile - Test if the end of file has been reached.
ReadAString - Read a string of characters from a file.
WriteAString - Write a string of characters to a file.
ReadAChar - Read a single character from a file.
WriteAChar - Write a single character to a file.
NextPrinting - Read the next character that is printable.

WriteALine - Writes a newline character to a file.
ReadACard - Reads a cardinal value in decimal form from a file.
WriteACard - Writes a cardinal value in decimal form to a file.
ReadALongInt - Reads a long integer in decimal form from a file.
WriteALongInt - Writes a long integer in decimal form to a file.
ReadALongHex - Reads a long integer in hexadecimal form from a file.
WriteALongHex - Writes a long integer in hexadecimal form to a file.

All read and write operations are buffered to improve IO performance.

Five predefined file handles exists for special input and output. These are:

StdIn - Standard input, normally keyboard.
StdOut - Standard output, normally VDU.
ErrOut - Error output, normally VDU.
AuxDev - Auxiliary device, normally serial port.
PrtDev - Printer device, normally parallel port.

The CommandLine object has an abstract state machine data structure and only has two associated operations (which are both selectors):

ReadArgument - Given the number of an argument on the command line it returns that argument. An argument is defined as a character string surrounded by whitespace.
NumberOfArguments - Returns the number of arguments on the command line.

The creator operation of this object is within the Modula-2 run-time system and is called when the program is run.

The Time object is very similar to the CommandLine object in that only selector operations are available within the Interface module. These are:

GetTime - Returns the current hours, minutes and seconds.
GetDate - Returns the current year, month and day.

The creator operation for this object is also within the Modula-2 run-time system and is presumably called at least once a second.

5.4.8. Listing File Control Module (Listing).

This module creates the listing file which is output from the meta-assembler. The listing file is implemented as an abstract state machine. It has two basic structures:

Page - The layout of a listing page is controlled by two operations: **SetPageLength** and **SetLineLength**. The first line of any listing page contains the page number, date and time. The second line contains the listing title, which is set by the **SetTitle** operation (the default is no title). The third and fourth lines of any listing page are always blank.

Line - Any line in the listing (apart from the first four on a page) contains four fields:

1. Line number, relative to the start of the listing file.
2. Address of the machine code instruction on that line (in hexadecimal format). Possible addresses range from 0 to 4GBytes.
3. Machine code. The machine code of the instruction on the line is given in hexadecimal form. The operation to write the machine code to the listing file is assembler-independent in that it adjusts itself to compensate for different word lengths and the maximum length of instructions.
4. The original assembly language statement follows the machine code field.

At the end of the listing file, assembly statistics are given for assembly time and assembly rate. Depending on the state of an internal switch (modified by the **SymbolTable** operation) a symbol table dump may/may not be included. The main part of the symbol table dump is performed by two operations exported from the table module (see section 4.8.10.). Similar switches control the inclusion of addresses and machine code.

When errors occur they are added to the listing file on separate lines to assembly language instructions and always follow the instruction to which they refer.

Other operations allow for the advancement of a page or line, and predicates allow tests for the following:

Whether a listing file is currently open.
The current source line being assembled in the source program.
The current line being written to the listing file.
The page and line length and the assembly rate in lines/minute.

5.4.9. Object Code Generator Module (ObjectGenerator).

This module is very similar to the listing module in design and functionality. Its purpose is to create the object file output by the meta-assembler. The format of this file is set with the **SetFormat** operation. Machine code instructions are added to the file with the **AddCode** instruction. Whether the object file is created or not is determined by the **ObjectOutput** operation.

5.4.10. Table Handler Module (Table, TableTrees, TableExt).

The table handler module allows for the storage and retrieval of symbols and opcode mnemonics. The decision to place opcodes in a table is justified in section 5.4.11. The table takes the form of an abstract state machine rather an abstract data type (for two separate tables) because this is a more memory efficient method which does not sacrifice speed (see below).

The decision to reuse a set of modules developed in the second year 'Programming and Program Design' course was made based on their extreme suitability for the task. Very little modification was necessary due to the object oriented approach taken during their design and implementation. The three modules implement a hash table, each location of which is a binary search tree. The combination of these data structures has several advantages:

1. Because binary search trees can be implemented using dynamic memory allocation, the table is limited in size only by available memory.

2. In an application such as this, the speed of entry to and retrieval from the table is important. The performance of tables, and other data structures, is difficult to measure. One method of estimating the performance of data structures, however, is by using "big O" notation.

<u>Big O Notation</u>.

Big O Notation measures the growth rate of the computation time taken to perform operations on a table against the number of entries in the table. Typical operations which are quoted for a table are: entry of an element, retrieval of an element, deletion of an element and the production of a report which contains all the elements in the table. For any particular implementation method, a big O value can be associated with each of these operations. Figure

29 shows the big O values for three possible methods of implementing a table.

The retrieve operation when performed on a table implemented as an unordered array, will take O(N/2) time, on average, to complete. This means that if there are N elements stored in the array, it will, on average, take N/2 searches to find that element. By contrast, a hash table implementation will take only one search, on average, to retrieve an item.

For the meta-assembler implementation, the most important performance time is that of retrieval. Symbols and opcodes will on average be retrieved more than once (whereas only one insertion is ever made). Symbols and opcodes are never deleted from the table. A report is only ever generated in the form of a symbol table dump at the end of the listing file. As can be seen in figure 29, the most suitable method of implementing the table, given these facts, is by hashing.

Hash Tables.

A hash table is usually stored as an array. The location *l* at which the record associated with the key *k* is stored is obtained by direct calculation of some function *f*:

$$l = f(k)$$

The performance of the implementation is determined by the choice of function f and the method of handling collisions.

Hashing Function.

A good hashing function will produce a uniform distribution of locations in a hash table given a random set of keys. There have been many suggestions for the implementation of the hashing function. The purpose of the hashing function in the case of the meta-assembler is to take an alphabetic key (either the mnemonic name or symbol name) and return an address within the array at which the data on the key will be stored. One of the simplest and most effective hashing algorithm is the division hashing algorithm. Characters in the key are multiplied by their positions in the string and added together. The total is then taken modulo the table size to give the position in the table.

h(K) = K MOD M where

K is the key to be hashed,
h(K) is the hashed key,
M is the size of the table.

Hashing Collisions.

The main drawback to hash tables is that collisions occur where two or more keys 'hash' to the same location in the table. Figure 30 shows how the performance of a hash table is drastically reduced when it becomes more than 80% full. There have been a number of proposals to the solution of this problem, some of which are:

* Linear Probing. When a collision occurs, one of the colliding data elements is put in the next available location in the table.

* Nonlinear Probing. A secondary hashing function is applied to any key which collides when hashed by the primary function.

* Bucket Hashing. Each location in the hash table can contain many data items. The data structure which provides this 'bucket' can be:

- Linked list.
- Binary search tree.
- Another hash table.

The method selected for the implementation is bucket hashing because it is less complicated than the other methods and can be dynamic which allows a hash table of any size (which stops the performance of the table from dropping drastically). In terms of overall performance, it is better to use a binary search tree when bucket hashing because they are quicker than linked lists (see figure 29) but are still dynamic (which hash tables are not).

Figure 31 gives the big O performance of the hashing table and binary search tree buckets combination. Performance on all functions is good except the report function where performance drops to $O(N^3)$. This, is still acceptable, however, as the operation is only used when symbol tables are being dumped (i.e., at most once in an assembly run). The other possibility would be to report without alphabetical sorting of the key items which would reduce big O value.

The three modules which implement the hash table are:

Table. This module implements the hash table as an abstract state machine. Operations on the table use the BST (Binary Search Tree) object which is implemented by the TableTrees module. All operations in this module are completely data independent (except that the key must be a string of characters).

TableTrees. This module implements a BST abstract data type and associated operations. The BST is exported opaquely and is therefore totally secure.

TableExt. This module (Table Extensions) contains the data structure which defines the data to be stored in the table and the operations on this data structure which are data-dependent. As two complete sets of data are required (one for instruction mnemonics and assembler directives, and one for symbols) the data structure used is a variant record. These allow for the most efficient storage of data. The data to be stored is:

For mnemonics and assembler directives:

* Name (Key).
* Type (or class).
* Machine code mask.

For symbols:

* Name (Key).
* Value.
* Status (Absolute or Relative).

This module provides two operations, **InsertCommand** and **InsertSymbol** which allow for the convenient insertion of either commands or symbols into the table. The advantage of using these operations is that a data structure does not need to be constructed to use them. They construct the appropriate data structure and then use the data structure-independent operation **Insert** (in the Table module) to enter them into the table. Operations are also supplied to write and read the data structure from and to a file, although these are not used

by the meta-assembler. The other two operations provided allow the conversion of a machine instruction mask from and to a readable format (again, these are not used).

5.4.11. Assembler Definition Module (ADM).

The purpose of the assembler definition module is to encompass all the assembler dependent data structures, parameters and algorithms. The two parts of the ADM, the definition and implementation modules, serve two purposes.

Definition Module.

This module describes the assembler to the rest of the modules via a number of parameters (Modula-2 constants). These are:

WORDLENGTH. The wordlength describes the length of a basic machine code instruction unit in terms of the number of bits required. Although the decision as to what constitutes a word in any processor is somewhat subjective, virtually all processors have an easily identifiable word length. Figure 32 gives a number of processors and their associated word length.

MAXINDICES. This constant describes the maximum number of indices allowable in an operand.

MAXOPERANDS. This constant describes the maximum number of operands allowable in an assembly language statement.

MAXWORDSPERINSTRUCTION. This constant describes the number of machine words in the longest machine instruction.

MEMLOW, MEMHIGH. These two constants describe the range which the processor can address. Depending on whether the assembler is specific to one computer, this can also describe the maximum memory physically possible on that computer.

NOOFREGISTERS. This constant is used to define an array which must be filled with the character string representations of all the registers that can be used in the assembly language being described. The array, ValidRegisters, is filled within the main body of the ADM implementation module.

Data structures to describe a machine word, instruction, all valid addressing modes, an operand element and a decoded line are included in the ADM definition module for convenience.

Implementation Module.

The ADM implementation module describes two operations which together form the algorithmic part of the specification for a processor.

InsertOpcodesInTable. This operation, as its name suggests, inserts all the assembler opcodes into the opcode/symbol table. This operation is called from the coordination (M2Assem) module. The operation uses the **InsertCommand** operation exported by the TableExt module to insert opcode manes into the table with their associated instruction mask and type. The type of the instruction is dependent on the layout of the instruction formed when using the opcode and is chosen by the programmer. It is used within the **Assemble** operation to select the appropriate algorithm to assembler the instruction.

Assemble. This operation forms the main part of the ADM implementation module and implements the code generator of a traditional dedicated assembler. The operation has an overall structure which is assembler-independent. This is shown in pseudo-code in figure 33. Each **Type** operation assembles a different format of machine instruction. As was shown in section 4.5., the regularity of a processors instruction set determines the number of instruction formats (and therefore **Type** operations) which are required. It follow that the length and complexity of the **Assemble** instruction depends to a great extent on the regularity of the instruction set of the processor being described.

The **Assemble** operation, and therefore each **Type** operation, is supplied with two parameters:

1. The data structure containing the decoded assembly language statement which is produced by the **ScanLine** operation in the lexical scanner module (Lex).
2. The current pass number (each **Type** operation used will be called twice, once for each pass).

The **Assemble** operation must produce two data items:

1. The machine code instruction corresponding to the decoded assembly language statement input.
2. The length of the above instruction.

The task which the **Type** operations must perform is dependent on which pass the meta-assembler is currently on.

In pass one the sole task is to determine the length of the machine code instruction. Depending on the processor being assembled for, this may simply involve returning the appropriate value (in which case pass one operation become trivial). However, with some processors the instruction is not constant and depends on the data length (modifier) and operands. In this case, more work is done which sometimes involves analysing operands.

In pass two, the **Type** operations perform the translation of decoded assembly language statement into machine language instruction. This process generally requires four steps (which may be in different orders or with some steps unnecessary).

1. The addressing mode of each of the operands in the assembly language statement must be determined (using the **ScanOperand** operation exported by the lexical scanner).
2. The number of operands and the correct addressing modes must be checked. If there is an error an exception must be raised with a suitable error message.
3. Each operand and index must be evaluated (using the **Evaluate** operation exported by the Expression module) or alternatively checked to see whether it is the appropriate register (using the **CheckRegister** operation, also exported from Expression). If an expression is not in the required range or a register is not of the correct type an exception should be raised.
4. Finally, the machine code instruction should be assembled. This may involve the conversion of operands into the appropriate format required.

Once the machine code instruction has been assembled, it can be returned together with its length. Control then passes back to the calling module (M2Assem in this case) for assembly to continue.

In order to test this design an ADM was developed for the Motorola MC68000 processor. A description of this design follows.

5.4.12. ADM for the Motorola MC68000.

The first task required in producing an ADM for the MC68000 was to provide values for the processor parameters.

WORDLENGTH = 16
MAXINDICES = 2
MAXOPERANDS = 2
MAXWORDSPERINSTRUCTION = 5
MEMLOW = 0
MEMHIGH = 1000 X 1024 (1 MByte)
NOOFREGISTERS = 17

The next task, turning to the implementation module, was to define all the possible registers. This was done by making assignments to the ValidRegisters array in the module body.

```
ValidRegisters[1] := "D0";
          :
          :
ValidRegisters[8] := "D7";
ValidRegisters[9] := "A0";
          :
          :
ValidRegisters[16] := "A7";
ValidRegisters[17] := "SP"
```

The **InsertOpcodesInTable** operation then needed to be filled with the appropriate **InsertCommand** operations. The problem is that the IEEE defined opcode set does not match one for one the Motorola defined opcode set, meaning

that one IEEE opcode can map onto several Motorola opcodes. The solution is to have a number of pseudo-Type operations which select the appropriate instruction format by examining the addressing modes of operands. Figure 34 shows all the IEEE standard mnemonics, the appropriate opcode mask, the type of the opcode and the corresponding Motorola opcode mnemonic(s). Note that the instruction types, as well as the standard 18, include an extra eight which correspond to IEEE standard opcodes which map onto several formats of instruction. The InsertCommand operations were then written from this table.

The other mapping which took place was from the IEEE standard addressing modes to the appropriate Motorola addressing modes. This can be seen in figure 35.

An operation, **RegisterType**, was is used to determine whether a register is a data or address register. Another operation allows the addition of a binary word at any position in another word. Many of the Motorola instruction formats include a field for a register and an effective address (see figure 6). Two operations were therefore provided to add a register and an effective address at any point within a machine instruction word.

The MC68000 is one of the processors for which the calculation of the length of an instruction is not always determined by its opcode. Pass one of the assembler is therefore more complicated in some **Type** operations.

For those instructions whose length is not always one word, the length is determined by the effective address field. Depending on the addressing mode, an instruction can be from one to five words long. The **CalcEA** operation determines the length of these instruction on pass one and assembles the effective address field on pass two.

The checking for the correct addressing mode, number of operands and number of indices is done within each **Type** operation, where necessary. The command modifier is also checked to determine whether an assembly language statement is to operate on byte, word or longword quantities.

The pseudo-**Type** operations use the **ScanOperand** operation to determine the addressing modes of operands which in turn determines which Motorola instruction format to choose. Calls to other **Type** operations occur within these operations.

The complete listing of the example ADM for the MC68000 is included in the appendix.

5.4.13. Coordination Module (M2Assem).

The coordination module drives the rest of the meta-assembler modules. Its structure is very similar to that of a dedicated two-pass assembler. The module contains two operations, **PassOne** and **PassTwo** which control each pass of the meta-assembler. The module opens and closes all the files used during assembly (input source program, output listing and object code files) by calls to the appropriate operations in the Interface, Listing and ObjectGenerator modules. The command line format which operates the meta-assembler is very simple:

M2Assem <filename>

The filename in the input file does not contain any extension. Default extensions are added to this file name to obtain the complete names of each file:

```
<filename>.asm     input assembly language program.
<filename>.obj     output object code file.
<filename>.lst     output listing file.
```

5.4.14. Assembler Directives Module (PseudoOps).

This module implements the assembler-independent assembler directives described in section 5.3.3. The directives have associated Type values above 1000. When the **Assemble** operation in the ADM cannot match the type to one of its **Type** operations it calls an operation in the PseudoOps module called **AssemblePseudoOp** which performs the same function as **Assemble**, except that it selects between the possible assembler directives.

5.5. Implementation.

Following the object oriented development philosophy, modules where implemented individually (as far as

possible). The order in which actual implementation of modules took place was determined to some extent by the way in which modules coupled with each other. For instance, it was necessary to implement the Strings, Exceptions and Table handling modules first so that testing of other modules could take place when they were developed.

5.6. Testing.

There were two levels of testing that were carried out during the development of the meta-assembler:

1. Individual Module Testing.
2. Module Integration Testing.

The first level of testing was carried out after the implementation of each module to ensure that the module behaved as expected. In order to carry out this testing three test modules were developed.

1. TestStrings. This module was used to test the operations exported from the Strings module to ensure that they were functionally correct.
2. TestLex. This module was used to test the functionality of the **ScanLine** operation exported from the lexical scanner module.
3. TestTable. This module was originally developed to test the three table handling modules: Table, TableTrees and TableExt. However, because the modules needed the table facilities in order to be tested properly, this module was developed further to incorporate tests for other modules as they were developed. The TestLex module's features were incorporated into this module later on so that a DecodedLine structure could be obtained.

The module presents the tester with a menu of possible operations:

1. A new symbol or opcode can be added to the table.
2. A symbol or command can be retrieved from the table.
3. A symbol or command can be deleted from the table.
4. An assembly language statement can be assembled. This must previously have been decoded by the lexical scanner.
5. The contents of the table can be printed out in order.
6. The number of entries in each hash location can be displayed.
7. An assembly language statement can be entered and then decoded by the lexical scanner.
8. An expression can be entered (which may contain previously defined labels) and evaluated by the expression

evaluator.

9. The default base of constants can be changed.
10. A listing file can be generated by entering an assembly language statement, its equivalent machine code and the number of times this is to be repeated.

This last test module therefore performs both individual module testing and to some extent integration testing. The real integration testing could only begin after the implementation of the coordination module, M2Assem. It is interesting to note that this was one of the last modules to be implemented.

Testing of individual modules using the three test programs actively involved the following test principles:

Internal Testing.

This is a process of comparing the design of modules and the way in which the code implements the design.

Complexity analysis - Examine complex areas of code to determine whether they are necessary or the result of bad design.

Structure analysis - Are all parts of the code called from all possible conditions? Are there any unnecessary conditions which are never executed? Do program structures have a single entry and exit point?

Internal testing was to some extent not so important as the program design and implementation was written by the same person. However, when using unsuitable methods of implementation it allowed the identification and correction of these areas.

External Testing.

This is involved in testing the implementation by regarding it as a box which should produce the correct output when given suitable input. The test process can be summarised:

1. Identify test data for the system from the specification.
2. Specify the expected results for each test case.
3. Run the program and record actual results.
4. Compare expected and actual results.

Implementation Method

Operation On Table	UNORDERED, SEQUENTIAL ARRAY	BINARY SEARCH TREE	HASH TABLE
Insert	O(1)	O(1)	O(1)
Retrieve	O(N/2)	O(Log(N))	O(1)
Delete	O(N/2)	O(N)	O(1)
Report	O(N x Log(N))	O(N)	$O(N^2)$

Figure 29. *Performances of Table Structures.*

The design of the test data determined how well the implementation could be tested. The following principles were used.

Equivalence Partitioning.

'The object of equivalence partitioning is to identify from the logic of the specification (program design) the categories into which input values are grouped'.

Boundary Case Analysis.

'An extension of equivalence partitioning which concentrates on the edges of the class'.

The testing both at individual and integration levels went well. The integration testing did highlight some areas of the design which, although not wrong, could definitely be improved (see Conclusions).

6. CONCLUSION.

6.1. Usefulness of Meta-Assemblers.

The usefulness of meta-assemblers has been proven by past implementations. Their use has evolved from their macro-based implementations in the 1960s which were used as a forerunner to the high level language, through the 1970s and 1980s when the development of a multitude of microprocessors lead to the need for a tool to develop and implement assemblers quickly and easily, right up to the present day. It is clear that the continuing development